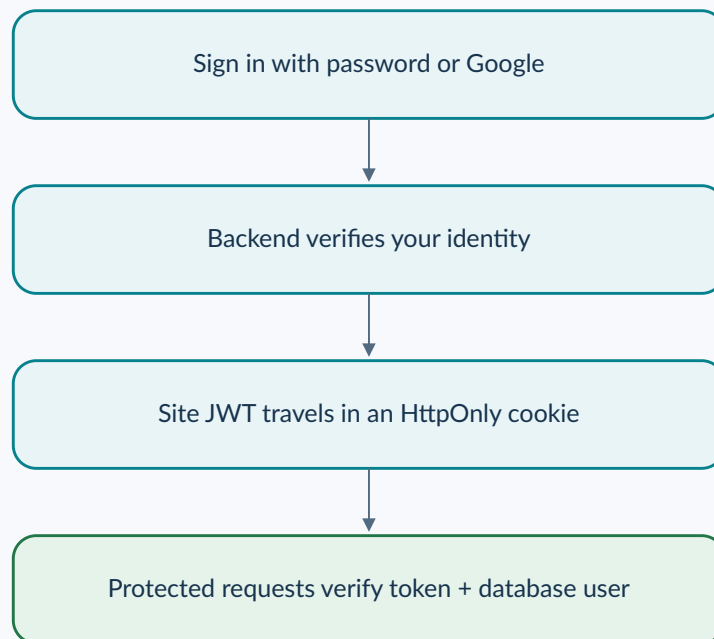


Understand your authentication

TRIVIA TRAP · COMPLETE ENGLISH EDITION

From your first request to a verified user.

A guided tour of the current backend/app/authentication folder. Written for someone learning backend authentication for the first time, with the actual names and decisions from your project.



The journey you will learn to explain, one step at a time.

What is inside

Flowcharts, foundational concepts, every Python function and class, database integration, browser behavior, complete examples, troubleshooting, and exercises.

Code snapshot: 6 October 2026 · Git revision c914099. **Reference:** Authentication_English_Flowcharts.pdf, all 27 pages reviewed. **Scope:** 10 Python modules, 29 function definitions, 9 classes, and the existing documentation/test HTML.

How to use this guide

Learn the journey first, then open the code beside the explanations.

You do not need to understand every Python line immediately. First learn what each request is trying to achieve. Then connect each step to a function. The function index near the end lets you jump directly to any definition.

Pages	Read this to understand
2–3	Start Here
4–12	The User Journeys
13–21	Foundations
22–42	Every File And Function
43–52	Integration And Practice
53–56	Check Your Understanding
57–62	Reference

Three labels used throughout

- **Current code:** behavior observed in the files in this checkout.
- **Illustration:** fictional users, tokens, times, or simplified code used to explain an idea.
- **Future work:** a useful design that the current code has not implemented.

Reading method

Follow one request with a pencil: who sends it, what is checked, what is stored, what is returned? Explain every failed branch as well as the successful branch. Use the exercises after the file walkthrough.

Passwords, tokens, client IDs, and addresses in examples are invented placeholders. No private .env values were read or copied into this guide.

What changed since your old PDF?

The old PDF is the visual reference; the present source is the behavior reference.

Topic	Older PDF	Current implementation
Account storage	TEST_USERS in memory	PostgreSQL users table through repository.py
User identity	Locally generated UUID	Database integer; JSON and JWT sub use a string
Sending the site token	Authorization: Bearer	Bearer header or access_token cookie
Browser storage	Not implemented in the archive	HttpOnly cookie; frontend fetch includes credentials
JWT claims	sub, iat, exp	sub, iat, exp, and ver
Password recovery	Not implemented	Random one-use reset token; digest stored in the database
Old sessions after reset	No revocation mechanism	Increment auth_version; earlier token versions fail
Main app integration	Not shown	main.py includes routers and credentialed CORS
Tests mentioned in docs	Historical information	backend/tests is absent in this checkout

A server restart no longer deletes registered users. A token can remain usable after restart when the same signing secret and database are retained and its time/version checks still pass.

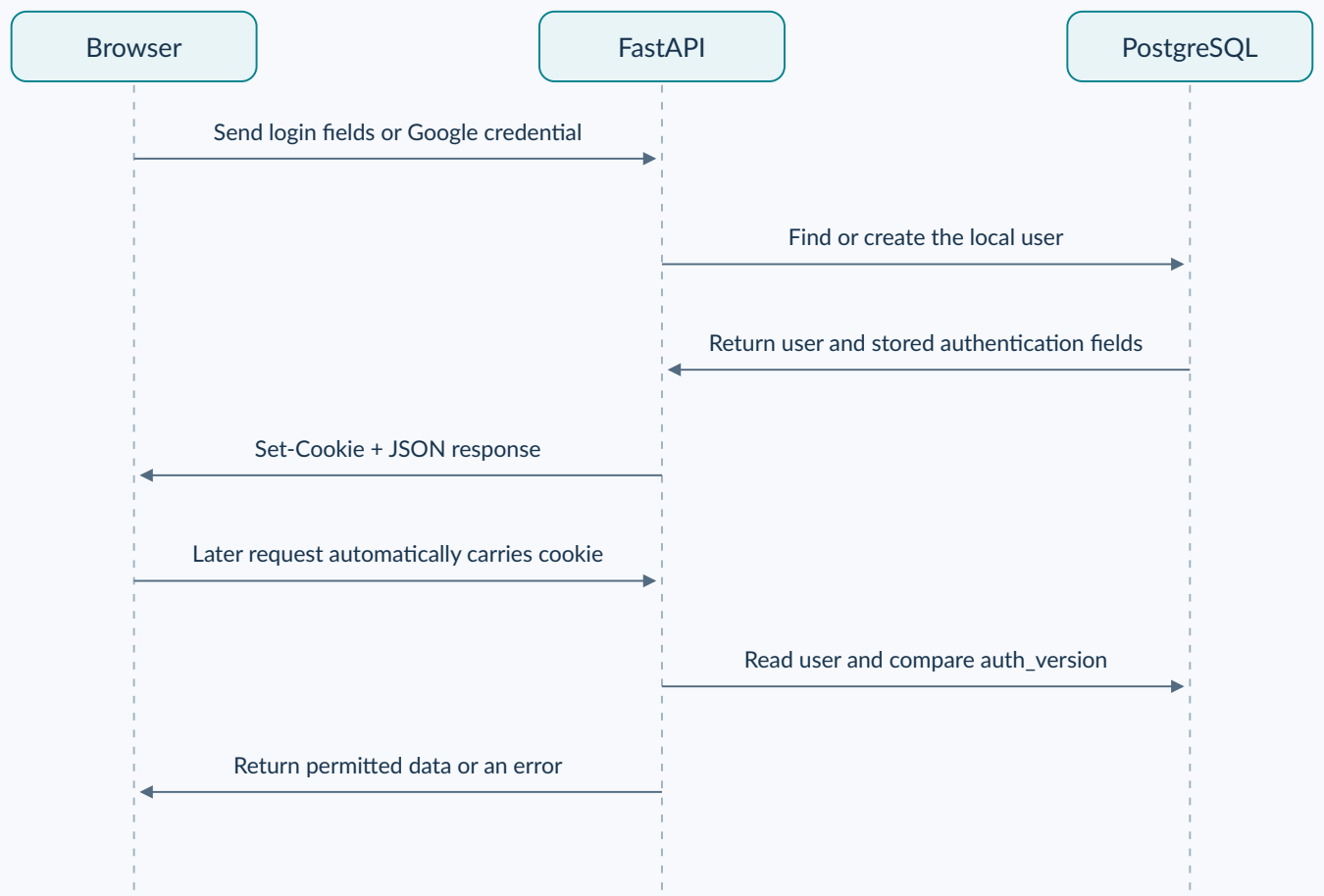
A detail that remains unchanged

Registration creates an account but does not sign the user in. The user still has to call /auth/login afterward. There is still no refresh-token or logout endpoint in this folder.

Code / evidence: Reference PDF; all current authentication modules; main.py; dataProcessing/models.py

Who does what?

Four participants work together; none does the whole job.



Participant	Responsibility
Frontend	Collect input, send requests, display errors, and choose which page to show.
FastAPI backend	Validate inputs, verify identity, enforce authentication, and produce responses.
PostgreSQL	Persist users, enforce uniqueness, and make reset updates atomic.
Google / email service	Google proves Google identity; Gmail SMTP transports reset and notification emails.

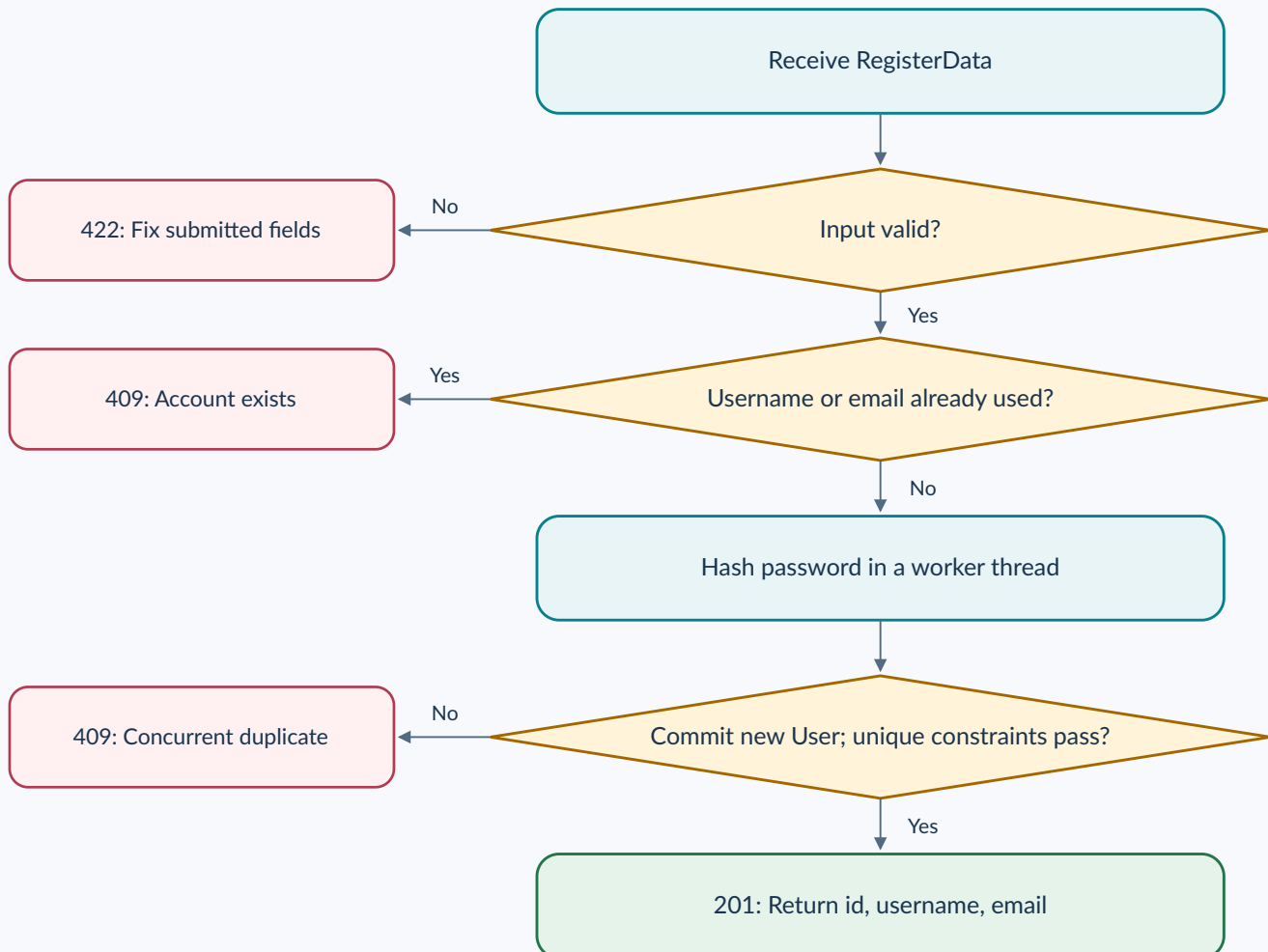
Authentication answers “who is making this request?” Authorization answers “is this user allowed to perform this action?” A valid token identifies a user; each feature still needs its own permission checks.

Code / evidence: router.py; current_user.py; repository.py; google_auth.py; mailer.py

1 • Create an account

POST /auth/register

Hamza enters a username, email address, and password. FastAPI validates them before the registration function uses them.



Read from top to bottom. A side arrow is a different outcome.

```
POST /auth/register
{"username": "Hamza42", "email": "hamza@example.com",
 "password": "HamzaLearning2026!"}
```

Why there are two duplicate checks

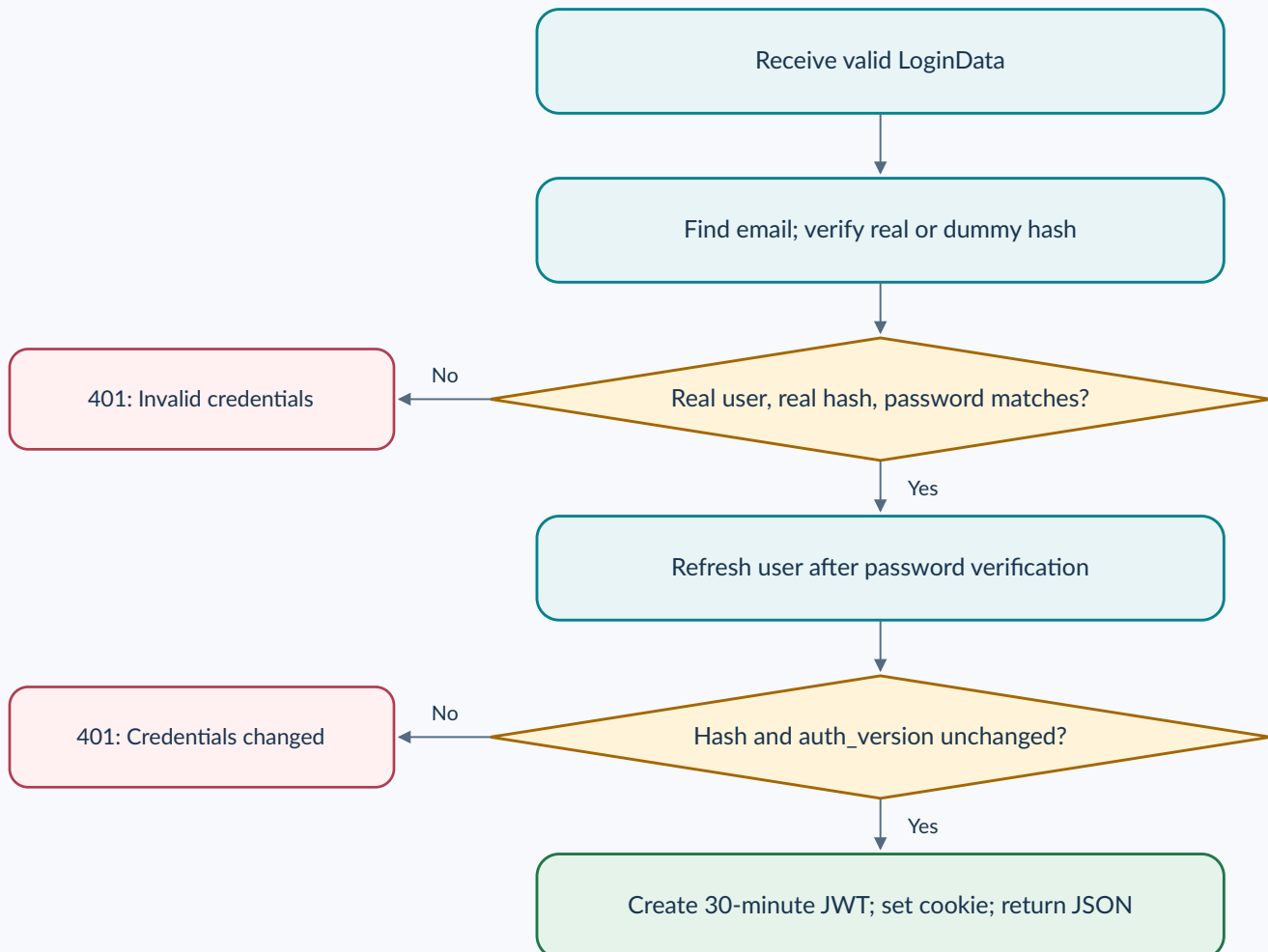
The lookup gives a quick answer. The database constraint also catches another request creating the same identity while hashing is running. No access token is created here.

Code / evidence: router.py: register; schemas.py: RegisterData; repository.py: create_user

2 • Sign in with a password

POST /auth/login

The password is checked against the stored password hash. The backend never decrypts the hash to recover the old password.



Read from top to bottom. A side arrow is a different outcome.

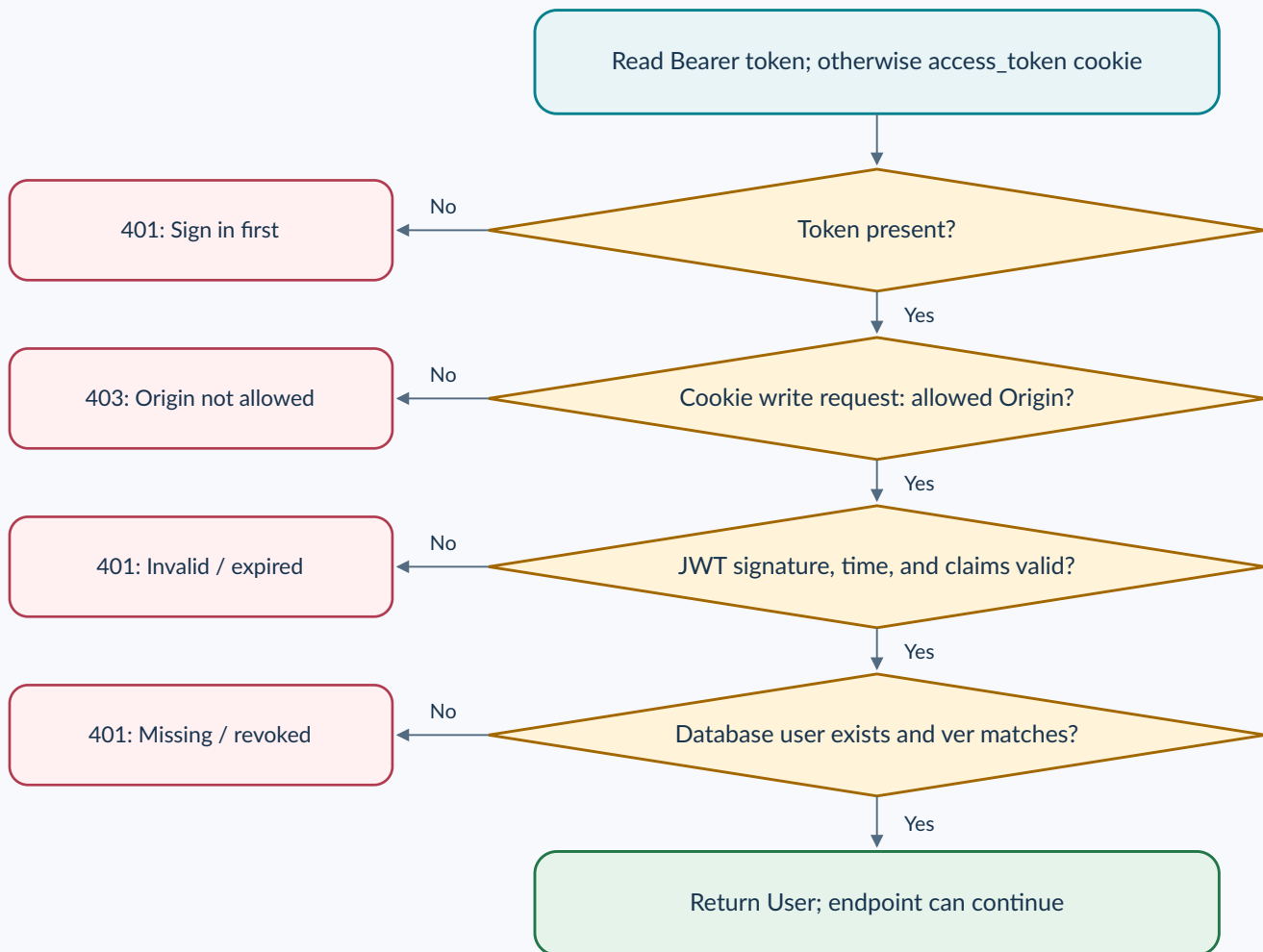
```
HTTP 200
Set-Cookie: access_token=SITE_JWT; HttpOnly; Max-Age=1800; ...
{"access_token":"SITE_JWT", "token_type":"bearer"}
```

The cookie and the JSON field contain the same site JWT. A malformed request gets 422 before this flow proceeds. After login, the frontend calls /auth/me to confirm the session works.

Code / evidence: router.py: login; security.py: authenticate_user; session_cookie.py

3 • Use a protected endpoint

GET /auth/me, or another endpoint that depends on `get_current_user`



Read from top to bottom. A side arrow is a different outcome.

The Origin step is required only when using the cookie for a method other than GET, HEAD, or OPTIONS. GET /auth/me skips that write check, but still verifies the token and database user.

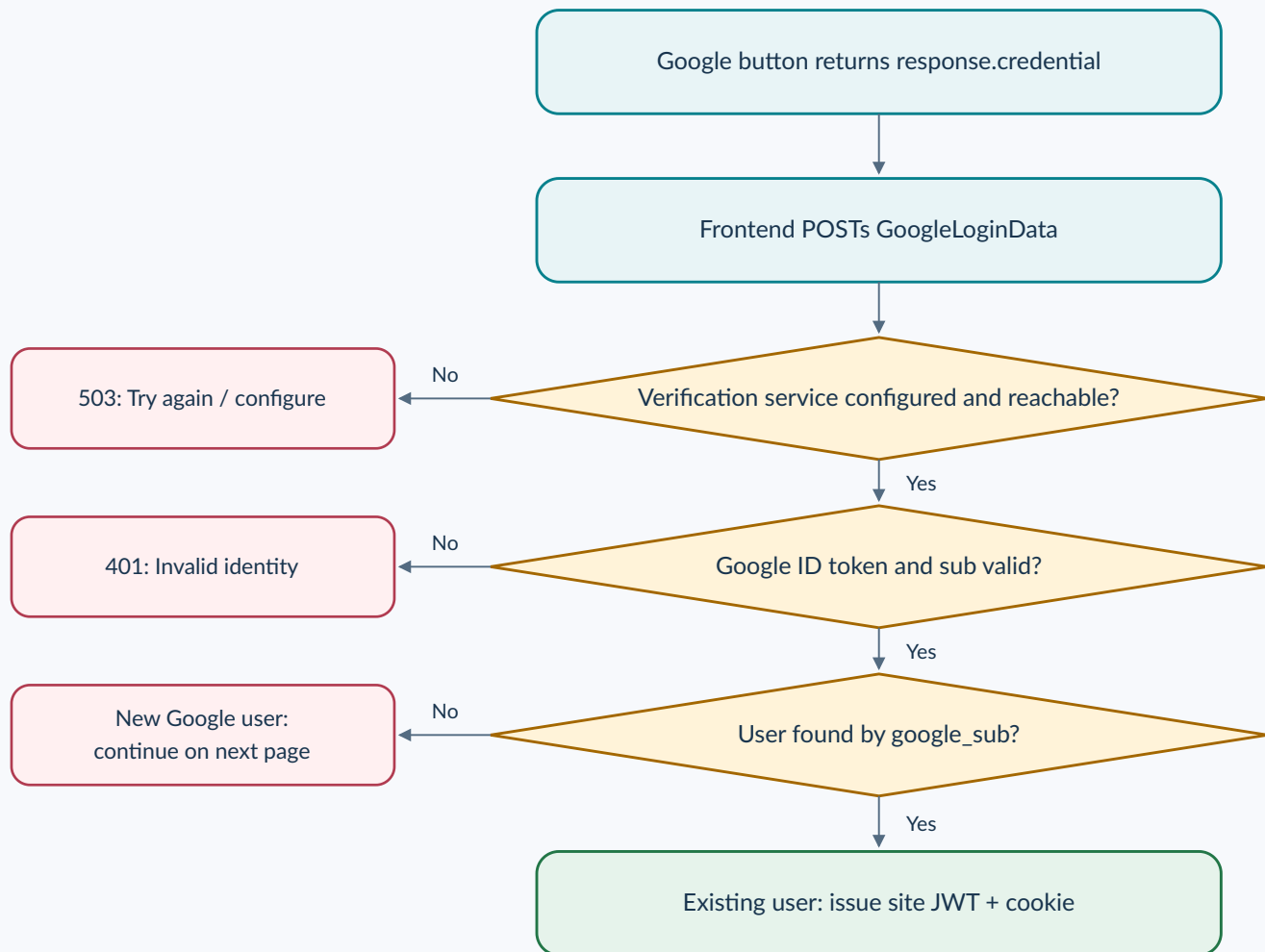
What /me returns

The `me()` handler returns `id`, `username`, and `email`. It does not issue another JWT or extend expiry. Every protected API request needs authentication even if the frontend already checked /me.

Code / evidence: `current_user.py: get_current_user`; `security.py: decode_access_token`; `router.py: me`

4 • Prove a Google identity

POST /auth/google • first verify, then find your local account



Read from top to bottom. A side arrow is a different outcome.

If that Google identity has no local user, continue with the new-user flow on the next page. Invalid request fields return 422.

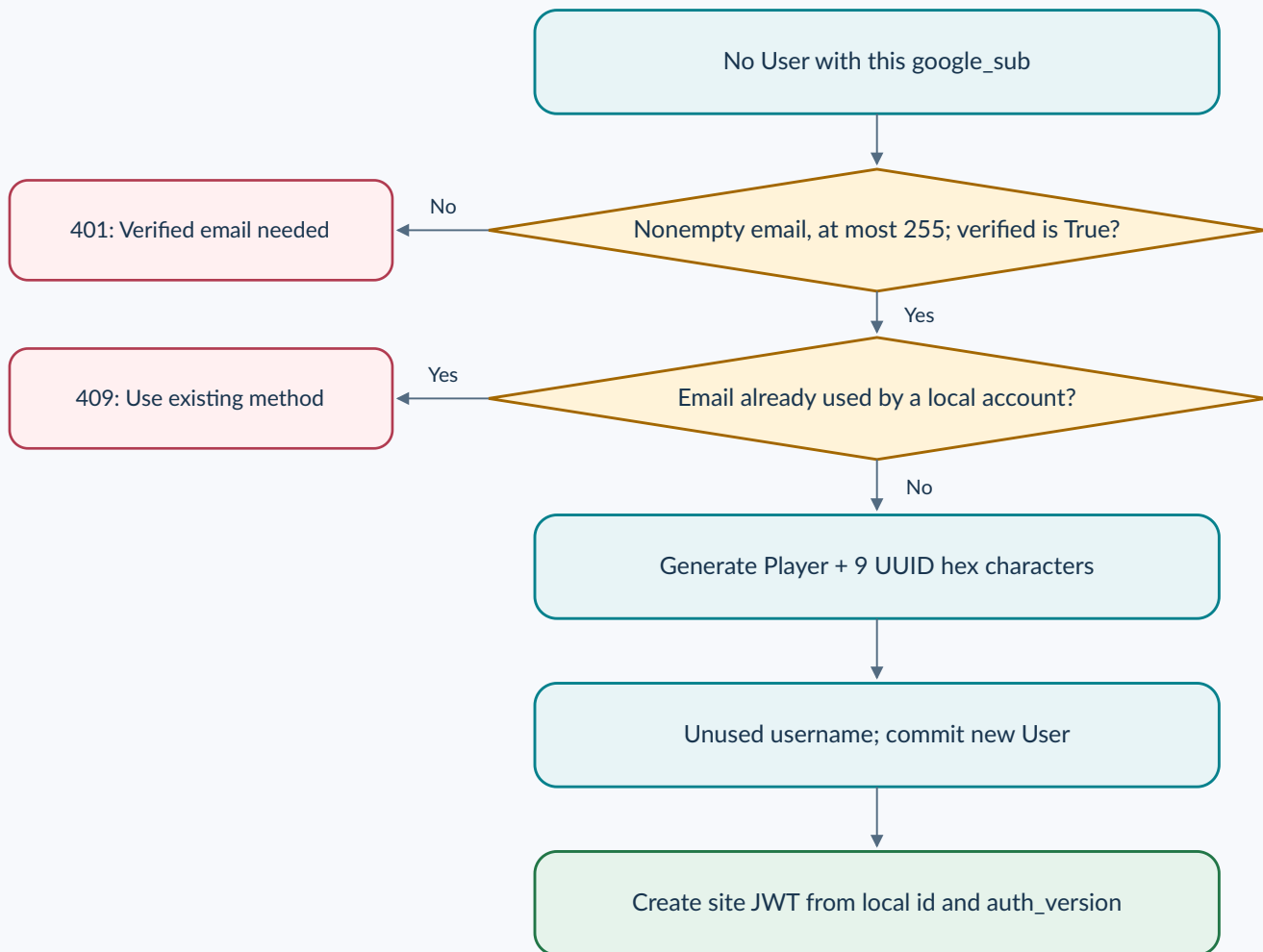
Google ID token	Site JWT
Issued by Google; proves a Google identity to your application.	Issued by this backend; identifies a local users row.
Submitted once for a Google sign-in attempt to /auth/google.	Sent with subsequent protected requests.

Google token verification checks include signature, intended audience, issuer, and expiration. [Google verification](#)

Code / evidence: google_auth.py: verify_google_token and google_login; frontend/google-sign-in.tsx

4b • A new Google user

Continue here only when google_sub was not found.



Read from top to bottom. A side arrow is a different outcome.

The username lookup repeats if a generated name is already taken. Database uniqueness still protects the final insert; an insert conflict is converted to 409. The account stores password_hash=None, so normal password login cannot authenticate it.

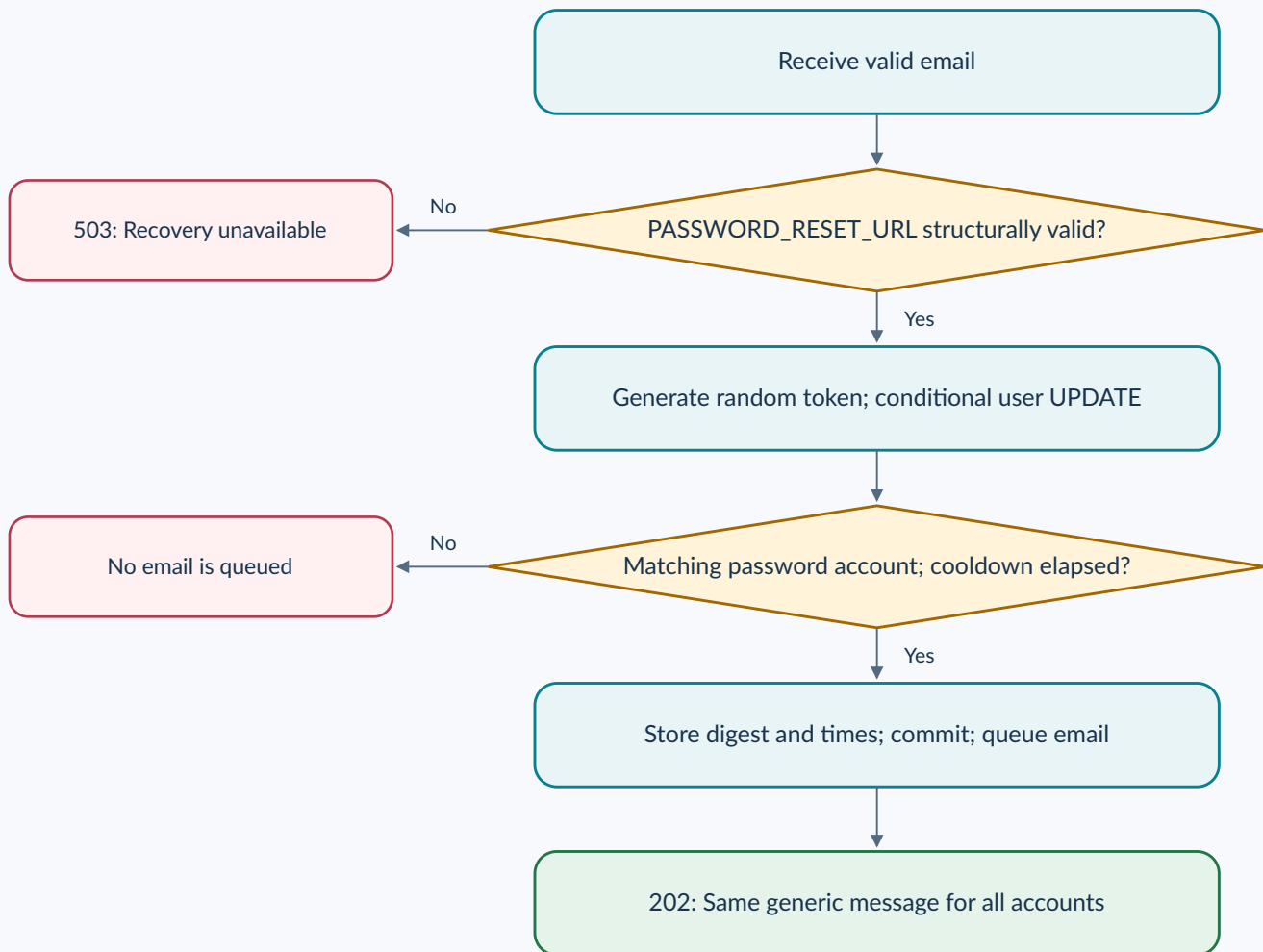
No automatic linking

Matching email addresses do not merge a password account and a Google identity. The current route refuses that collision. For an existing google_sub, the stored email is not refreshed by this function.

Code / evidence: google_auth.py: google_login; repository.py: find_user_by_username / create_user

5 • Ask for a reset email

POST /auth/forgot-password



Read from top to bottom. A side arrow is a different outcome.

The “No email” branch also ends with the same 202 response after committing the database operation. Missing accounts, Google-only accounts, and accounts within the 60-second cooldown all follow that branch.

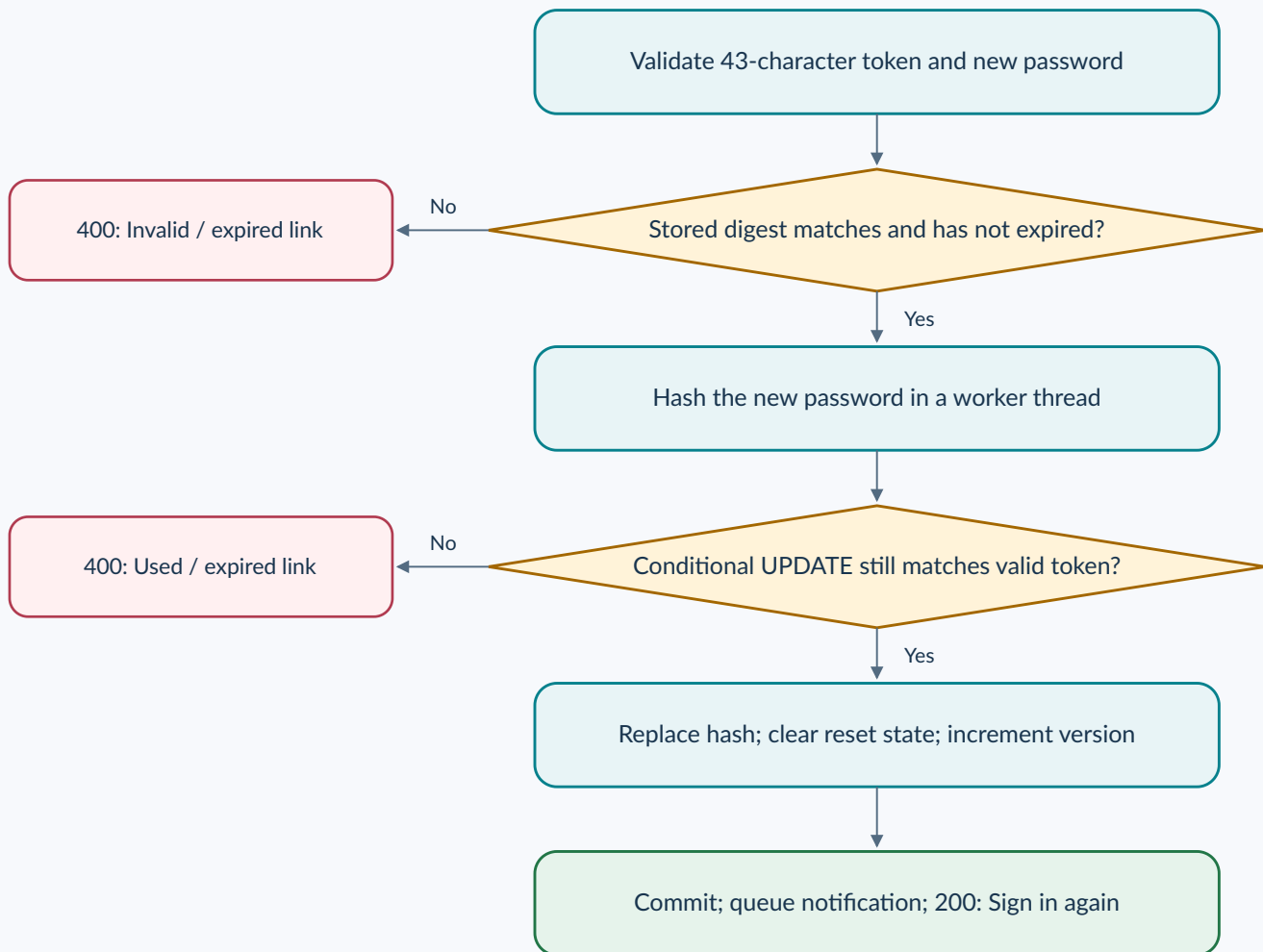
202 does not mean delivered

The HTTP response says the request was accepted. Sending happens later through BackgroundTasks, and delivery can fail. The email contains a token that expires after 15 minutes.

Code / evidence: password_reset.py: forgot_password; mailer.py: send_email

6 • Set a new password

POST /auth/reset-password



Read from top to bottom. A side arrow is a different outcome.

Invalid field formatting returns 422. The second database check happens after the expensive hash, because another request could use the token or time could pass while this request waits.

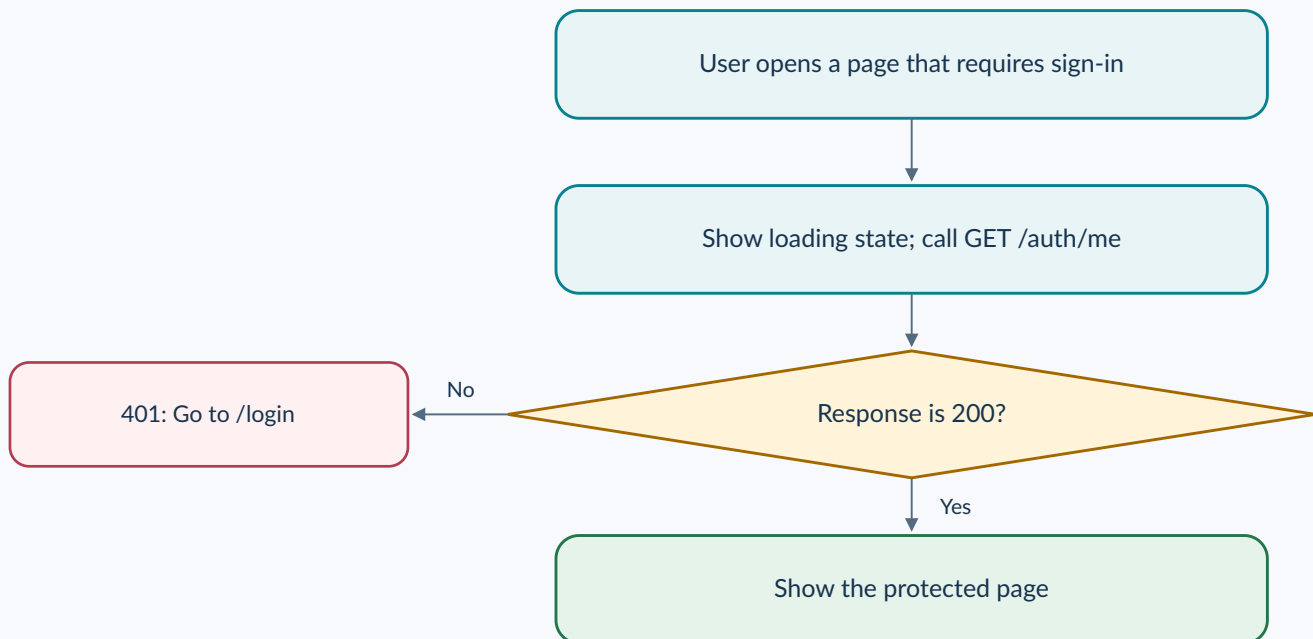
Why old logins stop working

Earlier site JWTs still contain their old ver claim. After the reset commits, `get_current_user` compares that claim with the higher database `auth_version` and rejects them. The browser may still show the old cookie until it expires or is removed.

Code / evidence: `password_reset.py: reset_password`; `current_user.py: get_current_user`

7 • Decide which page to show

Browser navigation and backend authentication solve connected parts of the job.



Read from top to bottom. A side arrow is a different outcome.

Illustrative protected-page pattern: treat a 401 as a missing/invalid session; treat a network error or server failure as a service problem and offer a retry. A failed connection does not prove that the password or token is wrong.

What the current frontend already does

The /login component checks /auth/me before showing its normal form. A successful response redirects to the local next path or /join. Other HTTP responses or a fetch error allow the form to display. The new-password mode bypasses this check so reset links can be used.

After either sign-in method, finishSignIn() calls /auth/me again, then navigates. apiFetch() includes cookies automatically. A shared guard for every future protected page is not present in this checkout.

The API remains the authority

A user can bypass browser navigation code and call an endpoint directly. Each protected backend route must depend on get_current_user and enforce its own permission rules. /join is the configured destination, but no /join page exists in this checkout.

Code / evidence: frontend/src/components/auth/login-page.tsx; frontend/src/lib/api.ts

Start with HTTP and JSON

A request asks the server to do something; a response reports the result.

```
POST /auth/login HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"email": "hamza@example.com", "password": "HamzaLearning2026!"}
```

Piece	Meaning in this example
POST	The HTTP method. Here it submits credentials for sign-in.
/auth/login	The route path. It selects the login operation.
Host	The server receiving the request. The port is part of its origin.
Content-Type	Tells the server that the body is JSON.
Body	The submitted fields, represented as structured text.
Status + body	The response might be 200 with a token, or 401 with an explanation.

A JSON object becomes a Python model such as `LoginData` after parsing and validation. A Python `UserResponse` model becomes a JSON response. You do not need to create a `.json` file for this exchange.

The routes exposed by this folder

Method and path	Success	Who can start it?
POST /auth/register	201 · account fields	A visitor
POST /auth/login	200 · JWT + cookie	A visitor with password credentials
POST /auth/google	200 · JWT + cookie	A visitor with a Google ID token
GET /auth/me	200 · account fields	A caller with a valid site JWT
POST /auth/forgot-password	202 · generic message	A visitor
POST /auth/reset-password	200 · message	A caller with a valid recovery token

HTTPS protects data during transport. Password hashing protects stored passwords. Both serve a purpose; one does not replace the other.

Code / evidence: `router.py`; `google_auth.py`; `password_reset.py`

Read the Python syntax without guessing

The patterns you keep seeing in these files.

Syntax	How to read it
<code>def / async def</code>	Define a regular function / a coroutine function. Async work must be awaited or scheduled.
<code>-> User None</code>	The result is a User object or no user. This annotation documents a type; it is not a database query.
<code>Annotated[T, ...]</code>	A type with extra metadata; FastAPI/Pydantic read the metadata here.
<code>Depends(function)</code>	Ask FastAPI to execute a dependency and supply its result.
<code>@router.post(...)</code>	A decorator that registers which HTTP route calls the following function.
<code>@field_validator(...)</code>	Tell Pydantic which field should run this validation method.
<code>@classmethod / cls</code>	The method receives the class, rather than a completed instance.
<code>self</code>	The particular instance whose method is running.
<code>*, google_sub=None</code>	Arguments after <code>*</code> must be passed by name.
<code>raise / return</code>	Stop by raising an exception / finish by giving the caller a value.
<code>from None</code>	Suppress the earlier exception context in a chained traceback.
<code>with / async with</code>	Manage a resource and clean it up when leaving the block.

Example: follow the arguments

```
async def me(
    user: Annotated[User, Depends(get_current_user)],
) -> UserResponse:
    return UserResponse(id=str(user.id),
                        username=user.username, email=user.email)
```

FastAPI supplies user after running `get_current_user`. The visitor cannot choose it by sending `{"user": ...}`. If the dependency raises 401, the successful body of `me()` never runs. [FastAPI dependencies](#)

Code / evidence: `router.py`; `repository.py`; `schemas.py`; `validation_route.py`

Passwords, hashes, and signatures

Three separate transformations appear in this project.

Operation	Purpose	Where you see it
Password hashing	Make guessing a stored password expensive; verification tests a candidate.	pwdlib in security.py
SHA-256 digest	Compare a high-entropy reset token without storing the raw token in users.	hashlib in password_reset.py
JWT signing / MAC	Detect changes to token contents and establish possession of the signing key.	HS256 in create_access_token

Password hashing, step by step

Registration receives a password as text, unwraps `SecretStr`, and asks `pwdlib` to hash it. The library uses a random salt and cost settings; the stored representation carries what verification needs. Your dependencies include `Argon2` support, and `PasswordHash.recommended()` selects the library's recommended hasher. [pwdlib guide](#)

```
stored = hash_password("HamzaLearning2026!")
verify_password("HamzaLearning2026!", stored) # True
verify_password("DifferentPassword123!", stored) # False
```

Two users with the same password can have different stored hashes because their salts differ. The verification function uses the stored hash's settings and salt. There is no "decrypt the hash" step.

Why SHA-256 is used only for the recovery token

A human password may be guessable. The recovery token is generated from 32 cryptographically random bytes, giving 256 random bits before encoding. A fast digest is suitable for matching that hard-to-guess token; it is not the password-storage algorithm. [Python secrets](#)

SecretStr has a smaller job

It masks normal display of a value. The original text still exists and `get_secret_value()` can retrieve it. `SecretStr` is not password hashing, encryption, or proof of identity.

Code / evidence: security.py; password_reset.py; schemas.py; backend/pyproject.toml

What is inside your site JWT?

A signed, readable entry pass with a time limit and a session version.

HEADER.PAYLOAD.SIGNATURE

```
# Illustrative decoded payload, not a usable token:
{
  "sub": "42",
  "ver": 3,
  "iat": 1800000000,
  "exp": 1800001800
}
```

Claim	Meaning in this implementation
sub	Local user ID as a string. "42" means users.id = 42.
ver	The user's auth_version when this token was issued.
iat	Issue time as an integer Unix timestamp, based on UTC.
exp	Expiry time, 1,800 seconds after issue.
nbf	Optional "not before" claim. The creation function does not add it.

The encoded header and payload can be read. HS256 creates an HMAC using JWT_SECRET_KEY, so changing sub or ver without recomputing a valid signature causes verification to fail. The payload should not contain secrets. [JWT specification](#)

decode_access_token() verifies the signature with a fixed allowed algorithm and requires all four claims. It then enforces additional claim types and ranges. get_current_user() separately checks that the database user exists and that the stored version matches.

Reading is not verification

A browser could decode text and look at exp, but it cannot establish your backend's identity decision just by reading it. It should call a protected API such as /auth/me. Never put JWT_SECRET_KEY into frontend code.

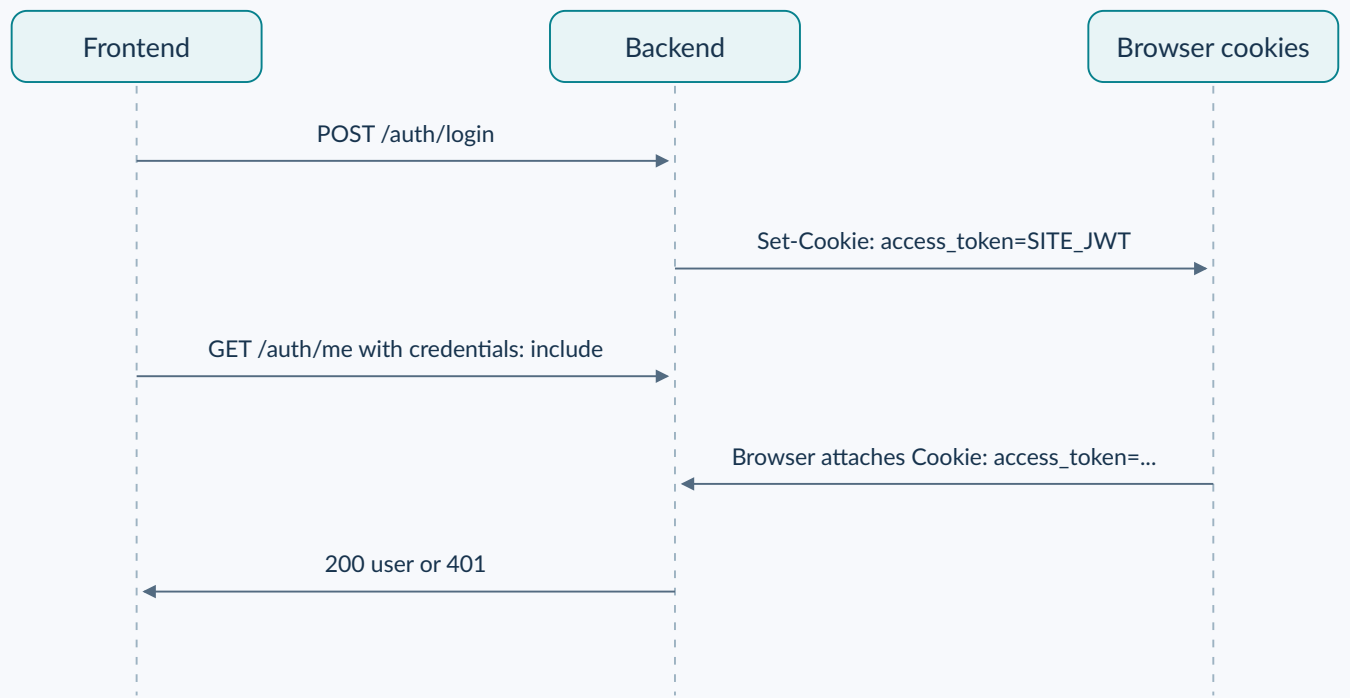
Calling /me does not renew the token. This folder has no refresh token flow. [PyJWT validation behavior](#)

Code / evidence: security.py: create_access_token / decode_access_token; current_user.py

JWT, cookie, session, and g_state

The value, its browser transport, and the signed-in experience.

A JWT is a token format. A cookie is browser storage that can be sent with matching requests. A session is the application's continuing recognition of a user. In your app, an HttpOnly cookie carries the JWT; the database stores the user and session version.



Setting	Meaning
HttpOnly	Page JavaScript cannot read this cookie through document.cookie.
Secure	Normally send over HTTPS. Your code permits HTTP only for the two configured loopback hostnames.
SameSite=Lax	Restricts sending in cross-site contexts; it is not the same rule as CORS.
Path=/; no Domain	Applies to paths under / on the host that set it. Cookies do not use ports as a boundary.
Max-Age=1800	The cookie is instructed to last 30 minutes. The JWT has its own exp check.

Google's g_state cookie tracks sign-out state for One Tap/automatic sign-in behavior. It is not used by get_current_user(). The app's Google button sets auto_select: false. No disableAutoSelect() call or logout endpoint is present, so this code does not notify Google whenever your site token expires. [Google state cookie](#)

Cookie behavior reference: [Set-Cookie](#). A JSON response containing the token remains readable to the requesting frontend even though the cookie copy is HttpOnly.

Code / evidence: session_cookie.py; current_user.py; frontend/src/lib/api.ts; google-sign-in.tsx

Hostnames, CORS, CSRF, and XSS

Understand exactly which protection applies where.

Concept	Concrete example	How this app uses it
Origin	http://127.0.0.1:3000 and :8000 are different origins.	main.py permits the two local frontend origins and credentials.
Cookie host	localhost and 127.0.0.1 are different hosts.	apiFetch adjusts a loopback API hostname to match the browser.
CORS	A browser decides whether frontend JavaScript may access a cross-origin response.	It does not verify user identity or constrain non-browser clients.
CSRF	A different website tries to trigger a write using a victim's automatically sent cookie.	The protected-request dependency checks Origin on cookie-authenticated writes.
XSS	Unwanted JavaScript runs within your site.	HttpOnly prevents reading the cookie value; scripts may still make requests as the user.

Follow the Origin condition literally

```
if credentials is None and request.method not in {
    "GET", "HEAD", "OPTIONS"
}:
    if request.headers.get("origin") not in ALLOWED_BROWSER_ORIGINS:
        raise HTTPException(403, "Origin is not allowed")
```

This runs inside `get_current_user`. It is not a global middleware rule. Public register, login, Google login, and reset routes do not call that dependency. A request with parsed Bearer credentials uses the header path and does not enter this cookie-specific Origin check.

Why this matters when adding routes

A new protected write endpoint needs the dependency. The code also needs feature permissions, such as “does this user own this record?” Session validation alone does not answer that. A missing Origin on a cookie-authenticated write is rejected too.

Code / evidence: main.py; current_user.py: `get_current_user`; session_cookie.py

The users row: what persists?

The authentication folder depends on `dataProcessing/models.py`.

Column	Purpose
<code>id</code>	Auto-generated integer primary key. JWT sub and response id turn it into text.
<code>username / email</code>	Display and contact values. Input schemas impose their own limits.
<code>username_key / email_key</code>	<code>casefold()</code> copies used for case-insensitive lookups and uniqueness.
<code>password_hash</code>	Password login verifier; None for a Google-only account.
<code>google_sub</code>	Google's stable subject identifier; optional and unique.
<code>auth_version</code>	Integer starting at 0. A password reset increments it.
<code>reset_digest</code>	64 hex characters from SHA-256 of a random recovery token. Indexed.
<code>reset_expires</code>	Timezone-aware timestamp limiting the recovery token's life.
<code>reset_sent_at</code>	When a reset was issued; supports the 60-second resend cooldown.
<code>created_at</code>	Database-generated creation time.
<code>avatar_url / cover_url</code>	Optional profile fields; not authentication credentials.

`User.normalize_identity()` runs when the ORM sets username or email. It also sets the corresponding normalized key, then returns the original display value. For example, Hamza42 becomes `username_key="hamza42"`.

Constraints provide the final rule

The database requires at least one authentication method: `password_hash` or `google_sub`. Unique normalized identity keys prevent two requests from registering equivalent names/emails. `google_sub` is also unique. Using SQL directly requires supplying normalized keys consistently because ORM validators do not run for every external SQL writer.

Different layers have different limits

The database username column allows up to 50 characters, while `RegisterData` accepts only 3–15. The registration API is intentionally stricter. The database is the persistent store; `DbSession` is a per-request database work context, not a login session.

Code / evidence: `dataProcessing/models.py`: `User` and `normalize_identity`; `repository.py`

async, await, and worker threads

The request waits for its result while the server can serve other requests.

```
password_hash = await run_in_threadpool(
    hash_password,
    data.password.get_secret_value(),
)
```

hash_password is synchronous and computationally expensive. run_in_threadpool receives the function and arguments and executes it in a worker. The coroutine waits with await; it does not finish registration until the result returns. [FastAPI async](#)

Work	How the current code runs it
User lookup and database writes	Async SQLAlchemy operations, awaited.
Password hashing / verification	Synchronous pwdlib functions dispatched with run_in_threadpool.
Google verification	Synchronous helper dispatched to a worker; may fetch Google certificates.
SMTP email	Synchronous send_email scheduled through BackgroundTasks after the response.

Waiting creates opportunities for other requests

Imagine two people registering “Hamza42.” Both can pass an early lookup before either finishes hashing. Only the database’s unique constraint decides which insert can commit. Likewise, password verification can overlap a password reset, so authenticate_user refreshes the stored user before accepting the earlier check.

Background does not mean guaranteed

BackgroundTasks runs work associated with the response in the application process. It is not a durable queue. A process failure can lose pending email, and send_email has no retry loop. [FastAPI background tasks](#)

A useful reading rule

An await marks a place where this coroutine may give other work a turn. Ask which assumptions could change before it resumes. A thread pool keeps blocking work off the event loop; it does not create unlimited capacity.

Code / evidence: router.py; security.py; google_auth.py; password_reset.py; mailer.py

Transactions, uniqueness, and atomic updates

Why checking once in Python is not enough under concurrency.

A transaction groups database work until commit or rollback. commit makes the successful write durable; rollback abandons the transaction's changes. A session coordinates this work for a request. SQLAlchemy does not turn every Python assignment into a committed database write immediately.

Expression in the code	What happens
<code>select(User).where(...)</code>	Build a query; it is executed when passed to the session.
<code>await db.scalar(query)</code>	Execute and return the first column of the first row, or None. For <code>select(User)</code> , that is a User object.
<code>db.add(user)</code>	Place a new ORM object in the session for insertion.
<code>await db.commit()</code>	Flush pending changes and commit the transaction.
<code>await db.rollback()</code>	Discard transaction work and restore a usable session after failure.
<code>await db.refresh(user)</code>	Read current stored column values back into the existing object.
<code>update(User).where(...).values(...).returning(User.email)</code>	Update only rows meeting the condition and return the changed email.

The reset token is consumed with one database operation

The final reset UPDATE checks identity, digest, and expiration at the same time as changing the hash, clearing the token, and increasing `auth_version`. Two requests cannot both consume the same stored digest successfully. A separate SELECT followed by an unconditional UPDATE would leave a race.

The boundaries are specific

The reset transaction protects token consumption and the version change. It does not undo API actions already authorized before the reset committed, and it does not make all other requests globally sequential.

General session behavior: [SQLAlchemy async ORM](#). The exact transaction decisions above come from your repository and reset functions.

Code / evidence: repository.py: create_user; password_reset.py: forgot_password / reset_password

The folder map

Read each file with one responsibility in mind.

File	Why it exists
router.py	Register the main routes and coordinate registration, password login, and /me.
schemas.py	Define accepted request shapes, public response shapes, and password/username rules.
security.py	Hash passwords, compare them, create and validate site JWTs.
repository.py	Centralize user lookups and committed account creation.
current_user.py	Turn request credentials into a verified User for protected endpoints.
session_cookie.py	Apply the same browser cookie policy to password and Google login.
google_auth.py	Verify Google ID tokens and map Google identities to local users.
password_reset.py	Issue recovery tokens, enforce cooldown, consume tokens, and revoke older sessions.
mailer.py	Deliver plain-text messages through Gmail SMTP.
validation_route.py	Remove submitted input values from authentication validation errors.
docs/database.md	Database setup and persistence notes; test references need the missing tests folder.
docs/password-reset.md	Recovery setup, API contract, timing, and limitations.
docs/index.html	Manual Google ID token acquisition page for developer testing.
docs/Authentication_*.pdf	The older English and Darija learning guides; the English one supplied the visual model.

The code reviewed here has no authentication/ `__init__.py`. Python can discover a namespace package when the parent directory is on its import path. Generated `__pycache__` files, when present, are interpreter caches and contain no separate source logic.

The next pages cover every defined Python function, including class validators and the nested `safe_handler`. The supporting database and frontend files are explained where they connect to authentication.

Code / evidence: Complete authentication directory inventory

schemas.py • the input and output contracts

Pydantic validates the shape before a route uses the data.

A BaseModel is a structured, validated object. Field adds limits; EmailStr validates and may normalize email syntax. It does not send a verification email. SecretStr masks normal representation and JSON serialization unless explicitly unwrapped. [Pydantic secret types](#)

Model / alias	Fields and constraints	Used by
RegisterData	email: EmailStr, max 255; username: 3–15; password: 15–128 with complexity.	register()
LoginData	email: EmailStr, max 255; password: 1–128; no new-password complexity validator.	login()
UserResponse	id: str, username: str, email: EmailStr.	register() and me()
TokenResponse	access_token: str; token_type defaults to "bearer".	login() and google_login()
Password	Annotated SecretStr with AfterValidator(validate_password_encoding).	Registration, login, and password reset.

Why the login password rule is different

Registration and reset choose a new password and enforce the full policy. Login checks an existing password against the stored verifier. Requiring a new-password policy there could reject a historically valid password before verification is attempted.

```
password_policy = (
    PasswordValidator()
    .has().uppercase().has().lowercase()
    .has().digits().has().symbols()
    .has().no().spaces()
)
```

The minimum and maximum password lengths come from Field, not from this policy chain. The chain checks categories and absence of spaces. A field validation failure raises a request validation error, which AuthRoute returns as a sanitized 422.

Response models keep the public contract small

UserResponse intentionally contains no password hash, reset digest, or Google subject. TokenResponse intentionally exposes the newly created site JWT. Extra arbitrary Python/database fields are not part of these declared response shapes.

Code / evidence: schemas.py: RegisterData, LoginData, UserResponse, TokenResponse, Password

schemas.py • all three validation functions

These functions reject bad input; they do not create users or sessions.

validate_password_encoding(value)

Receives: A SecretStr after basic type/field processing.

Steps: Unwrap the value and attempt UTF-8 encoding. Return the same object when encoding works.

Returns or raises: SecretStr, or ValueError with a safe message if UnicodeEncodeError occurs.

Why it exists: Reject malformed Unicode text, such as an unpaired surrogate, before a hashing library tries to encode it.

RegisterData.validate_username(cls, username)

Receives: A username that has already met the 3–15 character field length constraint.

Steps: Require username[0].isalpha() and username.isalnum(). Return the original text.

Returns or raises: The unchanged str, or ValueError.

Why it exists: Keep accepted names aligned with the application's letter-first, letters-and-numbers policy.

RegisterData.validate_password_complexity(cls, value)

Receives: The candidate SecretStr password.

Steps: Unwrap the password and ask password_policy.validate(...) to check required character categories and spaces.

Returns or raises: The original SecretStr, or ValueError describing the policy.

Why it exists: Share one complexity rule between registration and ResetPasswordData.validate_password().

Example	Result
Hamza42	Accepted username
42Hamza / Hamza_42	Rejected: first character / non-alphanumeric underscore
Élodie7	Letters can be Unicode; this is not an ASCII-only test
HamzaLearning2026!	Meets the shown new-password format rules

A validator does not store a password. get_secret_value() only unwraps text for the next operation. The password must still be hashed before persistence.

Code / evidence: schemas.py: lines 17–53

router.py • register()

The coordinator for creating an email/password account.

auth_router uses prefix="/auth", tags=["AUTH"], and route_class=AuthRoute. It includes google_router and reset_router, so their paths inherit /auth. main.py includes this combined router in the application.

register(data, db)

Receives: Validated RegisterData and an AsyncSession supplied through DbSession.

Steps: Look up any existing normalized username or email. On a conflict, raise 409. Otherwise hash the raw password through run_in_threadpool, call create_user(), and translate DuplicateUserError to 409. Construct a UserResponse.

Returns or raises: HTTP 201 with id converted to str, username, and email. Input validation returns 422 before the successful route body. Unexpected hashing/database failures are not converted into successful registration.

Why it exists: Combine input rules, password storage, and database creation into one public endpoint while leaving each lower-level responsibility in its own module.

```
user = await create_user(db, data.username, data.email, password_hash)
return UserResponse(
    id=str(user.id),
    username=user.username,
    email=user.email,
)
```

Who calls what?

FastAPI calls register. register calls find_existing_user_id, hash_password, and create_user. The repository commits the new User. FastAPI serializes the returned UserResponse.

Why the database still checks duplicates

The initial lookup and the final insert are separated by asynchronous work. Two attempts may both see “available.” A database unique violation becomes DuplicateUserError, which register turns into the same public conflict message.

Code / evidence: router.py: register; repository.py: find_existing_user_id / create_user

router.py • login() and me()

One creates a session credential; the other describes the authenticated caller.

login(data, db, request, response)

Receives: LoginData, a database session, the incoming Request, and FastAPI's outgoing Response object.

Steps: Unwrap the password; await authenticate_user. If it returns None, raise 401. Otherwise create_access_token(user.id, user.auth_version), call set_access_cookie, and return TokenResponse.

Returns or raises: HTTP 200 with JWT JSON and a Set-Cookie header. Failed credentials produce 401 with WWW-Authenticate: Bearer.

Why it exists: Give a verified local account the site credential used for later requests.

request supplies the URL scheme/hostname for cookie policy. response is modified to add a header; the route still returns a separate Pydantic TokenResponse as the JSON body.

me(user)

Receives: A User returned by Depends(get_current_user).

Steps: Build UserResponse from the verified object: id=str(user.id), username, and email.

Returns or raises: HTTP 200 account fields; authentication failure occurs in the dependency before this handler can complete.

Why it exists: Let the frontend confirm its session and obtain the current account without trusting a user ID supplied by the browser.

```
GET /auth/me
Cookie: access_token=SITE_JWT

# Illustrative success:
{"id": "42", "username": "Hamza42", "email": "hamza@example.com"}
```

It is a check, not a refresh

me() does not call create_access_token or set_access_cookie. Repeatedly calling it does not extend the 30-minute lifetime.

Code / evidence: router.py: login / me; current_user.py

security.py • password primitives

One hashing policy, two small functions, and a deliberate dummy verifier.

Initialization happens when the module is imported

JWT_SECRET_KEY is read from the environment. If absent or empty, RuntimeError stops startup instead of silently selecting a fallback secret. JWT_ALGORITHM is HS256 and ACCESS_TOKEN_EXPIRE_MINUTES is 30. PasswordHash.recommended() constructs the password hasher.

hash_password(password)

Receives: Plain password text, already validated by the request schema at its normal call sites.

Steps: Call password_hasher.hash(password).

Returns or raises: A hash string. It does not commit a User, set a cookie, or enforce the Pydantic password policy itself.

Why it exists: Keep the storage algorithm behind one function, used by register and reset_password.

verify_password(password, password_hash)

Receives: Candidate text and a stored hash string.

Steps: Ask password_hasher.verify to compare them. Catch UnknownHashError and return False.

Returns or raises: A boolean for matching/nonmatching/unknown hash format. Other unexpected errors are not caught by this except clause.

Why it exists: Give authenticate_user one consistent password comparison operation.

DUMMY_PASSWORD_HASH

The module computes a hash once from a fixed dummy password. authenticate_user uses it when an email is absent or the account has no password hash. The expensive verification still runs, reducing an obvious timing difference between “unknown account” and “wrong password.” It does not guarantee identical total timing.

The dummy password cannot sign someone in

After verification the code still requires a real user, a real password_hash, and a match. Knowing the dummy input does not bypass those checks.

Code / evidence: security.py: module settings, hash_password, verify_password, DUMMY_PASSWORD_HASH

security.py • create and verify JWTs

The names are short; the verification rules are intentionally specific.

create_access_token(user_id, auth_version)

Receives: A database user ID integer and the user's current session-version integer.

Steps: Read UTC now. Build sub as str(user_id), ver, iat, and exp=now+30 minutes. Call jwt.encode with JWT_SECRET_KEY and HS256.

Returns or raises: The signed JWT string. It does not set its own cookie or query the database.

Why it exists: Make password and Google sign-in issue the same kind of local credential.

decode_access_token(token)

Receives: A site JWT string from get_current_user.

Steps: Use jwt.decode with the secret, algorithms=[HS256], and required sub/iat/exp/ver. Convert TypeError, ValueError, or OverflowError into InvalidTokenError; library token exceptions can propagate. Apply the additional checks in the table.

Returns or raises: A tuple (int(user_id), version), or InvalidTokenError. The caller decides the HTTP response.

Why it exists: Centralize cryptographic verification and prevent malformed claims from reaching user lookup.

Additional check	Exact rule in your function
Timestamp types	iat, exp, and optional nbf must have type int; bool is rejected.
sub shape	A string; ASCII decimal digits; at most 10 characters.
sub range	1 through 2,147,483,647, compatible with the expected database ID range.
ver	Exactly an int, at least 0; bool is rejected.

The function does not require a site aud or iss claim. The fixed signing secret and algorithm are its current trust configuration. Checking that the user still exists and that ver is current is the next function's responsibility.

Token verification/time behavior: [PyJWT](#). Exact type/range rules: security.py.

Code / evidence: security.py: create_access_token / decode_access_token

security.py • authenticate_user()

Password verification includes a second look at the stored user.

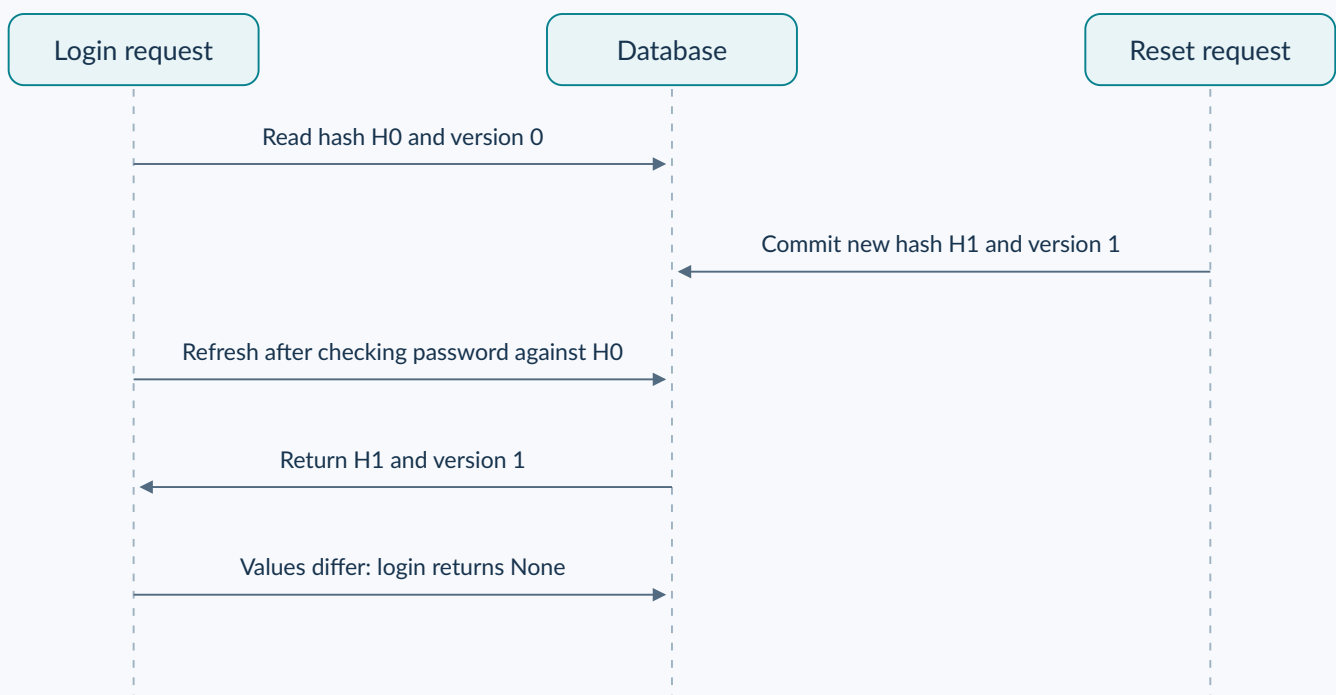
authenticate_user(db, email, password)

Receives: An AsyncSession, validated email text, and unwrapped password text from login().

Steps: Find the user by normalized email. Remember their password hash and auth_version, or a missing-user default. Verify the candidate in a worker against the real hash or dummy hash. Reject if the user/hash/match is missing. Refresh the User from the database, then compare the new hash and version with the remembered values.

Returns or raises: A User only if credentials match and stayed consistent; otherwise None. login() maps None to 401. Unhandled database/library problems can still be errors.

Why it exists: Authenticate credentials and detect a password reset that committed while the expensive password check was running.



Understand the race protection precisely

The refresh catches changes visible at that recheck. A reset that commits after the recheck still advances auth_version; a JWT issued from an older version will fail on the next protected check. This is not a global lock over every request.

The ordinary /auth/login route uses this function. Google login proves identity with Google's token instead, then shares the JWT/cookie creation helpers.

Code / evidence: security.py: authenticate_user; router.py: login; password_reset.py: reset_password

repository.py • all five lookup functions

Database queries shared by the authentication routes.

`DbSession = Annotated[AsyncSession, Depends(get_db)]` tells FastAPI to inject a database session. The annotation is a reusable dependency declaration, not a global connection or a signed-in browser session.

Function	Receives / query	Returns / reason
<code>find_user_by_id(db, user_id)</code>	Reject IDs outside 1..2,147,483,647; otherwise await <code>db.get(User, user_id)</code> .	User or None. A primary-key lookup for token authentication; invalid range avoids querying.
<code>find_user_by_email(db, email)</code>	<code>select(User) where email_key equals email.casefold()</code> .	User or None. Used by password authentication and Google-email collision checks.
<code>find_user_by_google_sub(db, google_sub)</code>	<code>select(User) where google_sub equals the given exact string.</code>	User or None. Identifies returning Google accounts independently of display email.
<code>find_user_by_username(db, username)</code>	<code>select(User) where username_key equals username.casefold()</code> .	User or None. Checks generated Google usernames.
<code>find_existing_user_id(db, username, email)</code>	<code>select(User.id); normalized username OR email match; limit(1).</code>	int ID or None. Registration needs to know whether either identity is already taken.

Trace one query

```
select(User).where(User.email_key == email.casefold())

# Conceptually, with a bound value:
SELECT * FROM users WHERE email_key = :normalized_email;
```

The SQLAlchemy expression binds the value as data. The repository does not build SQL by joining raw user input into an SQL string. `casefold()` supports caseless matching beyond simple ASCII lowercasing; it is not email delivery verification or a Gmail alias-normalization rule.

Lookup does not authenticate

Finding a row proves only that a record matches. Password verification, Google verification, or JWT verification must establish the caller's identity. None means no match, while a database connection failure may raise an exception.

Code / evidence: `repository.py`: `DbSession` and `find_*` functions

repository.py • create_user()

The place where account creation becomes persistent.

create_user(db, username, email, password_hash, *, google_sub=None)

Receives: An AsyncSession, account display fields, a hash or None, and optional keyword-only Google subject.

Steps: Construct User, which triggers ORM identity normalization. Add it to the session. Await commit. On IntegrityError, rollback; if the original PostgreSQL SQLSTATE is 23505, raise DuplicateUserError, otherwise re-raise the database error.

Returns or raises: The committed User object, with its database-generated ID. DuplicateUserError identifies a uniqueness conflict to the route layer.

Why it exists: Let password registration and Google registration share persistence and conflict handling.

```
try:
    await db.commit()
except IntegrityError as exc:
    await db.rollback()
    if getattr(exc.orig, "sqlstate", None) == "23505":
        raise DuplicateUserError from exc
    raise
```

DuplicateUserError is a domain-level exception

The class only subclasses Exception; pass adds no methods. The repository raises it to mean “account uniqueness conflict.” Routes translate it into HTTP 409, so low-level database details are not the public API contract.

Why rollback is needed

A failed transaction cannot simply be treated like a successful write. Rolling back discards failed work and resets the session’s transaction state. Only the 23505 uniqueness case becomes DuplicateUserError; a different constraint or infrastructure error is not mislabeled a duplicate.

How the result stays usable

AsyncSessionLocal is configured with expire_on_commit=False in dataProcessing/database.py. This lets normal loaded attributes remain available after commit. The repository returns the User so the route can construct its response.

Code / evidence: repository.py: DuplicateUserError / create_user; dataProcessing/database.py

current_user.py • the authentication gate

Use this dependency wherever an API operation needs a signed-in user.

unauthorized()

Receives: No arguments.

Steps: Construct HTTPException with status 401, detail="Could not validate credentials", and WWW-Authenticate: Bearer.

Returns or raises: The exception object. The caller must use raise unauthorized() to stop execution.

Why it exists: Keep missing, expired, invalid, and revoked credential responses consistent.

get_current_user(credentials, db, request)

Receives: Parsed HTTPBearer credentials or None, DbSession, and incoming Request.

Steps: Prefer the parsed Bearer token. Otherwise read request.cookies[access_token]. Reject a missing token. For cookie-based writes, enforce the allowed Origin list. Decode the JWT; translate InvalidTokenError to 401. Find the database user by ID and require the version to match.

Returns or raises: The User for the endpoint; 401 for missing/bad credentials or missing/mismatched user; 403 for a disallowed Origin on the cookie write path.

Why it exists: Give protected endpoints a trusted account object without duplicating authentication logic.

What HTTPBearer(auto_error=False) does

FastAPI's helper parses Authorization: Bearer ... into a credentials object. auto_error=False lets this code decide how to handle absent usable Bearer credentials and fall back to a cookie. credentials.credentials is the token string without "Bearer".

Header precedence can explain a surprising 401

If a parsed Bearer token is supplied, that token is used even when a valid cookie also exists. An invalid header token does not trigger a retry using the cookie. Remove the stale header when diagnosing the browser-cookie path.

This dependency identifies the user; it does not assign admin roles or grant access to someone else's records. Apply authorization in the protected feature.

Code / evidence: current_user.py: all module definitions

session_cookie.py • set_access_cookie()

One browser-session policy for both sign-in methods.

set_access_cookie(response, request, token)

Receives: The outgoing Response, incoming Request, and newly created site JWT.

Steps: Compute secure=True when the request uses HTTPS or its hostname is not localhost/127.0.0.1. Call response.set_cookie using COOKIE_NAME, token, max_age, httponly, secure, samesite, and path.

Returns or raises: None. Its result is a Set-Cookie response header, which tells the browser to store the token.

Why it exists: Avoid different cookie flags or lifetimes between ordinary login and Google login.

```
response.set_cookie(
    key="access_token",
    value=token,
    max_age=30 * 60,
    httponly=True,
    secure=secure,
    samesite="lax",
    path="/",
)
```

Constant / setting	Why it is here
COOKIE_NAME	Shared by the setter and get_current_user's reader.
ALLOWED_BROWSER_ORIGINS	http://localhost:3000 and http://127.0.0.1:3000. Used by CORS and the protected cookie-write Origin check.
No explicit domain	Cookie defaults to the host that set it; changing ports alone does not change the cookie host.
Lifetime	Uses ACCESS_TOKEN_EXPIRE_MINUTES from security.py, so the cookie policy follows the JWT setting.

Frontend JavaScript should not look for this value in document.cookie. It asks /auth/me, and fetch includes credentials. A production frontend origin must be configured deliberately; the existing origin list contains local development addresses only.

Setting and deleting are different operations

This module only sets the cookie. There is no logout route that deletes it in the current authentication folder. A future logout endpoint would need to expire the matching cookie; revoking other copies is a separate decision.

Code / evidence: session_cookie.py: set_access_cookie; router.py / google_auth.py call sites

google_auth.py • verify_google_token()

Google proves an identity; your backend still owns the site session.

GoogleLoginData and google_router

GoogleLoginData is a BaseModel with credential: SecretStr and min_length=1. It checks that a value was submitted, not whether Google signed it. google_router uses AuthRoute and is included by the main authentication router.

verify_google_token(credential)

Receives: The unwrapped Google ID token string from google_login().

Steps: Read GOOGLE_CLIENT_ID. If absent, raise 503. Call id_token.verify_oauth2_token(credential, GoogleRequest(), audience=client_id). Map TransportError to 503 and ValueError/GoogleAuthError to 401.

Returns or raises: The verified claims dictionary, or HTTPException indicating unavailable verification or invalid credentials.

Why it exists: Ensure the credential was issued by Google for this application before trusting its identity claims.

Check / name	What it means
Google signature	The token has a valid signature under Google's verification keys.
audience / aud	The ID token is intended for your configured Google client ID.
issuer / iss	The token issuer is Google, as checked by the Google library.
expiration / exp	The credential has not expired.
GoogleRequest()	The Google library HTTP transport helper; different from FastAPI Request.

Google subject sub identifies the Google account. Your own JWT sub is a separate local database ID. The Google verification helper runs in a worker because it is synchronous and can make network requests. [Google verification](#)

Configuration versus secrets

GOOGLE_CLIENT_ID is a public application identifier; the frontend and backend must agree on it. JWT_SECRET_KEY and SMTP_PASSWORD are private backend secrets. This flow does not use a Google client secret or perform an authorization-code exchange.

Code / evidence: google_auth.py: GoogleLoginData and verify_google_token

google_auth.py • google_login()

Map a verified Google identity into the same User model as password accounts.

google_login(data, db, request, response)

Receives: GoogleLoginData, DbSession, Request, and Response.

Steps: Verify the credential in a worker. Require sub to be a nonempty string no longer than 255. Look up google_sub. If absent, require a nonempty email string of at most 255 characters and email_verified is True; refuse any existing email; generate an unused username and create the user with password_hash=None. Finally create the site token and set its cookie.

Returns or raises: HTTP 200 TokenResponse and Set-Cookie. Bad identity/email can cause 401; conflicting email or insert uniqueness causes 409; verification configuration/transport can cause 503.

Why it exists: Make Google authentication compatible with every endpoint that already trusts the site JWT.

```
# Illustrative new row
User(
    id=73,                                # assigned by PostgreSQL
    username="Player1b2c3d4e", # Player + nine hex characters
    email="player@example.com",
    google_sub="GOOGLE_SUBJECT",
    password_hash=None,
    auth_version=0,
)
```

The returning-user shortcut

A User found by google_sub proceeds directly to site-token creation. The new-user email requirements and email collision branch are not rerun, and the stored email is not updated. The verified Google subject remains the identity key.

What account creation deliberately avoids

There is no automatic linking to an existing password account, no stored Google ID token, and no Gmail permission grant. EmailStr validation is not applied to the Google claims before this User creation; the code performs its explicit type, length, and verified-flag checks.

Google notes additional email-ownership considerations for non-Gmail, non-Workspace addresses. This implementation does not branch on hd. [Google verification guidance](#)

Code / evidence: google_auth.py: google_login; repository.py; session_cookie.py

password_reset.py • input rules and constants

Recovery is its own proof of account control.

Definition	Current meaning
ForgotPasswordData	One EmailStr email field, maximum 255 characters.
ResetPasswordData	token: SecretStr, exactly 43 characters; password: Password, 15–128 characters.
RESET_SECONDS	$15 \times 60 = 900$ seconds before a reset token expires.
RESEND_SECONDS	60 seconds between eligible reset issues for one account.
GENERIC_MESSAGE	The same accepted-request message regardless of eligibility.
reset_router	Uses AuthRoute and gets /auth from the parent router.

ResetPasswordData.validate_token(cls, value)

Receives: The candidate token wrapped in SecretStr, after its exact-length constraint.

Steps: Unwrap it. Require ASCII and every character to be alphanumeric, hyphen, or underscore.

Returns or raises: The same SecretStr or ValueError("Invalid reset token").

Why it exists: Reject malformed tokens before any digest lookup. Correct formatting alone does not establish validity.

ResetPasswordData.validate_password(cls, value)

Receives: The candidate SecretStr password after the shared Password/length processing.

Steps: Call RegisterData.validate_password_complexity(value).

Returns or raises: The value or the reused validator's ValueError.

Why it exists: Keep reset and registration complexity requirements consistent.

Two different kinds of invalid token

A 20-character string fails the schema and produces 422. A correctly shaped 43-character random string can pass the schema but fail the database digest/expiry check, producing 400. Neither outcome grants authentication.

Code / evidence: password_reset.py: constants, ForgotPasswordData, ResetPasswordData

password_reset.py • forgot_password()

Issue recovery state without revealing whether an account is eligible.

forgot_password(data, background_tasks, db)

Receives: Validated email data, a FastAPI BackgroundTasks collector, and DbSession.

Steps: Validate PASSWORD_RESET_URL first for every request. Generate UTC now and secrets.token_urlsafe(32). Run a conditional UPDATE for the normalized email, a non-null password_hash, and no recent reset_sent_at. Store the token digest, expiry, and issue time; return the row's email. Commit, then queue an email only if a row matched.

Returns or raises: HTTP 202 with the same GENERIC_MESSAGE, or 503 for bad reset URL configuration. Schema failures produce 422.

Why it exists: Allow account recovery while reducing disclosure of missing, Google-only, or cooldown-limited accounts.

Configuration validation is explicit, but limited

urlsplit must produce an http/https scheme and a hostname. The configured URL may not contain a fragment or embedded username/password. Parsing ValueError marks it invalid. These checks do not contact the URL or prove the frontend is running. The destination is configuration, never an untrusted request Host header.

```
UPDATE users
SET reset_digest = :digest, reset_expires = :expiry,
    reset_sent_at = :now
WHERE email_key = :email_key
    AND password_hash IS NOT NULL
    AND (reset_sent_at IS NULL OR reset_sent_at <= :cooldown_cutoff)
RETURNING email;
```

This is illustrative SQL for the SQLAlchemy operation. A single conditional UPDATE coordinates simultaneous requests for the same row. After the cooldown, a later eligible issue replaces the previous digest, so only the latest issued link can match.

Email failure does not undo the issue

Commit happens before scheduling/sending. If SMTP fails, the stored cooldown and reset state remain. A 202 response therefore cannot be used as evidence that an email arrived.

Code / evidence: password_reset.py: forgot_password

password_reset.py • reset_password()

Verify, hash, recheck, and consume the proof in one committed update.

reset_password(data, background_tasks, db)

Receives: A validated recovery token and new password, BackgroundTasks, and DbSession.

Steps: SHA-256 the unwrapped token. SELECT an eligible user ID by digest and future expiry. If absent, raise 400. Hash the password in a worker. UPDATE that same ID only if digest and expiry still match; replace password_hash, clear reset_digest/reset_expires, and add 1 to auth_version. On no returned email, rollback and raise 400. Otherwise commit and queue a change notification.

Returns or raises: HTTP 200 with the sign-in-again message, 400 for invalid/expired/used state, or 422 for input violations.

Why it exists: Consume the recovery token once and invalidate older site JWTs as part of the same database write.

```
.values(  
    password_hash=password_hash,  
    reset_digest=None,  
    reset_expires=None,  
    auth_version=User.auth_version + 1,  
)
```

What changes, and what does not

The raw token is never written into the users table. The digest and expiry are cleared after success, while reset_sent_at remains to enforce the recent-issue cooldown. No new JWT is created and no cookie is refreshed or explicitly cleared by this route. The user must log in with the new password.

Why two database checks are useful

The first SELECT avoids expensive hashing for obviously unusable links. The second conditional UPDATE is the decisive check because hashing takes time. It prevents the first result from being treated as a permanent reservation.

Only a future expiration is accepted: reset_expires > the current UTC time. At the expiry boundary the token is no longer eligible.

Code / evidence: password_reset.py: reset_password; security.py: hash_password

Why simultaneous reset requests stay safe

Trace the shared database state instead of imagining one request at a time.

Time	Request A	Request B	Stored state
1	Finds valid digest D	Finds valid digest D	digest=D; ver=0
2	Hashes password A	Hashes password B	Both are still waiting on work
3	Conditional UPDATE succeeds	May reach the UPDATE too	A locks/changes the same row
4	Commits hash A; clears digest; ver=1	Cannot match D after A commits	digest=None; ver=1
5	Returns 200	No returned email; rollback; 400	Only A consumed the token

PostgreSQL coordinates the competing row updates; the second request cannot consume the cleared digest. If the first transaction fails and rolls back, a later attempt may still succeed while the original token remains valid. “Only one success” does not mean “the first request always wins.”

Session revocation follows from the version comparison

```
Earlier JWT: {"sub":"42", "ver":0, ...}
Current row: users.id=42, auth_version=1

get_current_user():
    user.auth_version != version # True -> 401
```

The token's signature may still be correct and exp may still be in the future. It is rejected because its version is old. New password login receives ver=1. Increasing the one per-user version invalidates all earlier versions, rather than one selected device.

Do not overextend the guarantee

A reset does not erase a copied token from another device. It changes whether future validation accepts it. It also does not roll back actions that already passed their authentication check. The dummy hash, refresh check, unique constraints, cooldown UPDATE, and consume UPDATE each address a different problem.

Code / evidence: password_reset.py; security.py: authenticate_user; current_user.py

mailer.py • send_email()

A small SMTP adapter used by both recovery messages.

send_email(to, subject, body)

Receives: A recipient address, subject text, and plain-text message body from a scheduled background task.

Steps: Read SMTP_EMAIL and SMTP_PASSWORD. If either is missing, log a configuration error and return. Build EmailMessage fields and text content. Connect through SMTP_SSL to smtp.gmail.com:465 using a default SSL context and a 10-second timeout; log in and send the message.

Returns or raises: None. SMTPException or OSError is caught and logged by exception class name. Those failures do not become a new HTTP response after the original request already returned.

Why it exists: Keep mail transport separate from token issuance and password-change logic.

```
background_tasks.add_task(
    send_email,
    email,
    "Your Trivia Trap password was changed",
    "Your password has been reset. Sign in with your new password.",
)
```

This is a simplified call showing the roles of the arguments; the actual route includes additional advice in its body. add_task receives the function itself and arguments. Calling send_email(...) immediately would perform the work at that point instead of scheduling it.

Dependency	What it contributes
EmailMessage	Formats From, To, Subject, and the text body.
ssl.create_default_context()	Creates standard TLS verification settings for the SMTP connection.
smtpplib.SMTP_SSL	Opens the encrypted SMTP connection from the beginning.
logger.error	Reports missing configuration or the exception type without including the reset token or password.

Delivery limitations visible in this file

The SMTP provider and port are fixed. There is no retry loop, durable job store, or delivery confirmation returned to the frontend. A successful password reset remains committed even if its notification cannot be sent.

Code / evidence: mailer.py: send_email; password_reset.py: background_tasks.add_task

validation_route.py • safe validation responses

A wrapper around FastAPI's route handler.

AuthRoute subclasses APIRoute. The authentication routers use it so submitted passwords and tokens are not echoed through the normal request-validation error payload.

AuthRoute.get_route_handler(self)

Receives: The route instance created by FastAPI.

Steps: Call `super().get_route_handler()` to capture the original request handler. Define an inner `async safe_handler` that closes over it. Return that inner function.

Returns or raises: `Callable[[Request], Awaitable[Response]]`: a function that takes a `Request` and can be awaited for a `Response`.

Why it exists: Insert one consistent error-handling step around each authentication route.

safe_handler(request)

Receives: An incoming `Request` when FastAPI executes the wrapper.

Steps: Await `original_handler(request)`. If `RequestValidationError` occurs, loop over `exc.errors()` and retain only `loc`, `type`, and `msg`. Return `JSONResponse` with status 422 and detail set to that filtered list.

Returns or raises: The original response on success, or sanitized 422 JSON for request validation errors.

Why it exists: Exclude the input field that might contain a password, credential, or even the entire submitted body.

```
{ "detail": [{
  "loc": ["body", "username"],
  "type": "string_too_short",
  "msg": "String should have at least 3 characters"
}] }
```

Its scope is deliberately narrow

HTTPException such as 401/409 is not `RequestValidationError` and follows the framework's normal handling. This wrapper is not a global sanitizer for arbitrary logs, exception messages, or response validation errors. Exact validation wording can vary by dependency version.

Code / evidence: `validation_route.py`: `AuthRoute`, `get_route_handler`, nested `safe_handler`

docs/index.html • the manual Google test page

A developer utility, separate from the real Next.js login screen.

The page asks for a public Google client ID, loads the Google Identity Services script, and displays a Google ID token in a textarea. It asks you to copy that credential into the backend /docs interface. There is no fetch call to /auth/google in this file.

Function / callback	What it receives and does	Why it exists
googleLoaded()	No explicit arguments. Enables the initialize button and updates status text.	Lets setup begin only after the Google script loaded.
googleFailed()	No explicit arguments. Replaces status with a loading-failure message.	Makes a script/network failure visible.
setup submit handler(event)	Calls preventDefault, reads/trims client ID, checks its suffix, and initializes Google with a callback. Renders the button, disables initialize, and makes the client ID read-only.	Keeps the form from navigating away and prepares a stable manual test.
Google callback(response)	Reads response.credential, puts it into the token textarea, and reports success.	Makes the Google ID token available for manual submission to the backend.

The submit handler's `endsWith(".apps.googleusercontent.com")` check is only a basic format test. Google configuration and backend audience verification still have to match. The script element's `onload/onerror` attributes call `googleLoaded/googleFailed`; the actual sign-in library is external.

The other files in docs/

- **database.md:** documents PostgreSQL setup, migrations, normalized uniqueness, persistent reset/version state, and a proposed integration-test command.
- **password-reset.md:** documents mail/reset URL configuration, the two request bodies, the fragment-token frontend contract, cooldown, expiry, and delivery limits.
- **Existing English/Darija PDFs:** historical teaching documents. The supplied English PDF describes the older in-memory design. Use the updated source walkthrough for current behavior.

Documentation versus files on disk

Both Markdown documents mention backend/tests and tests that would mock Google/SMTP. That folder is absent in this snapshot. This guide does not claim those tests were available or executed.

Code / evidence: docs/index.html; docs/database.md; docs/password-reset.md; reference English PDF

The supporting backend files

Authentication connects to application startup, database sessions, and migrations.

File / definition	What it does for authentication
app/main.py	Creates FastAPI, configures CORS with allowed local browser origins and credentials, and includes auth_router and friends_router. root() handles GET /; it does not validate a session.
dataProcessing/database.py	Creates an async engine and AsyncSessionLocal bound to a hardcoded PostgreSQL URL. expire_on_commit=False preserves loaded attributes after a commit. Base is the shared declarative ORM base class.
get_db()	An async generator: enter a session context, yield the session to the request dependency, then close it on exit. Repository/reset write functions commit explicitly.
dataProcessing/models.py: User	Defines stored user columns and relationships. normalize_identity(key, value) sets the casefold key and returns the original assigned value.
Authentication migration 6724f643c277: upgrade()	Adds identity keys, auth_version, and reset columns/index. Backfills normalized keys from existing rows; makes them non-null and unique. Conflicting normalized identities fail instead of silently merging.
Same migration: downgrade()	Drops those added constraints, index, and columns. It reverses schema additions and loses that column data; it is not a password-recovery action.
backend/Makefile	make run uses uv run --env-file .env fastapi dev app/main.py --port 8000. That is how environment settings reach the process in local development.

The migration filename mentions authentication and friendship, but this migration's visible upgrade function adds the authentication columns above. A migration records a schema change; changing models.py alone does not update an existing database.

Database sessions and login sessions are different

get_db supplies a temporary database work context. An access_token represents a user's continuing authentication. Closing an AsyncSession at the end of a request does not log the browser out.

Code / evidence: main.py; dataProcessing/database.py / models.py; alembic/versions/6724f643c277_*.py; Makefile

Why these imports exist

Group the tools by the job they do.

Library / symbol	Role in your code
FastAPI: APIRouter, Depends, Request, Response	Register routes, provide dependencies, read incoming request metadata, and set response headers.
HTTPException / status / JSONResponse	Produce explicit HTTP failures and the sanitized 422 response.
Pydantic: BaseModel, Field, EmailStr, SecretStr	Validate fields and declare request/response structures; mask sensitive display.
Annotated / AfterValidator / field_validator	Attach validation and dependency metadata to types and fields.
pwdlib / PasswordValidator	Hash and verify passwords / apply new-password category rules. These are different libraries with different jobs.
jwt (PyJWT)	Encode and validate the local signed token.
SQLAlchemy: select, update, or_, AsyncSession	Build and execute database queries and transactions.
google.auth / google.oauth2.id_token	Verify the Google ID token and classify Google transport/authentication failures.
datetime / timedelta / timezone	Represent UTC issue/expiry times and time intervals.
secrets / hashlib	Generate a recovery token securely / calculate its SHA-256 digest.
uuid4	Generate a random suffix for a Google username; no longer the local user ID generator.
os.environ / urlsplit	Read process configuration / parse the configured recovery-form URL.
smtplib / ssl / EmailMessage / logging	Connect to SMTP with TLS, format mail, and report failures.
Callable / Awaitable	Describe the wrapper function type in validation_route.py.

Type hints make the interfaces easier to read and check. FastAPI and Pydantic interpret some of those hints at runtime. A plain Python return annotation by itself does not verify a password, perform a database lookup, or enforce permissions.

Code / evidence: Imports from all 10 authentication modules; backend/pyproject.toml

How the real frontend connects

These helpers explain the browser behavior you asked about.

File / function	Role in the current integration
lib/api.ts: isLoopback()	Recognizes localhost and 127.0.0.1.
lib/api.ts: apiBaseUrl()	Uses NEXT_PUBLIC_API_BASE_URL or port 8000; aligns a configured loopback API hostname with the browser's loopback hostname.
lib/api.ts: apiFetch()	Requires a path starting with one slash, then calls fetch with credentials: "include".
login-page.tsx: safeNextPath()	Accepts a local slash path, rejecting leading // or /\. Defaults to /join. It does not verify that the destination exists.
LoginPage session effect	Checks /auth/me before the normal login form. Success redirects; failure allows the form. Cleanup ignores late responses after unmount.
captureToken() in reset effect	Reads #token= from the URL into a ref, clears the fragment with history.replaceState, and listens for hash changes.
finishSignIn()	Removes a legacy localStorage token, confirms /auth/me succeeds, then navigates to the next path.
handleGoogleCredential()	POSTs the ID token as credential; handles errors and calls finishSignIn.
changeMode()	Switches login/register/reset views and clears password/message state when allowed.
handleSubmit()	Checks confirmation/reset-token formatting, chooses the endpoint/body, sends the request, and maps outcomes to messages/navigation.

GoogleSignIn loads accounts.google.com/gsi/client, initializes the callback with popup UX and auto_select:false, and renders a button. onReady/onError update loading state. The callback gives handleGoogleCredential the Google token; the component does not create a site JWT.

Why the reset form loses its token on reload

The raw recovery token is kept in a React ref after the URL fragment is removed. A full reload loses that memory. Reopen the original email link to supply the token again.

Code / evidence: frontend/src/lib/api.ts; components/auth/login-page.tsx / google-sign-in.tsx; app/login/page.tsx

Google sign-in, OAuth, and OIDC

Learn the names without adding imaginary steps to your code.

OAuth 2.0 is an authorization framework: a person can grant an application permission to call another service. OpenID Connect adds a standardized identity layer and ID token. Sign-in and permission to read Google data are different decisions. [OpenID Connect](#)

Term	What to understand here
ID token	A provider's statement about a user, intended for a particular application. Your credential from Google is an ID token.
Access token	A credential accepted by an API. Your backend calls its locally issued JWT access_token.
Refresh token	A credential that may obtain new access tokens. This backend has no refresh-token endpoint.
Client ID	Public identifier of the Google application configuration. The frontend ID should match the backend audience.
Scope	A named permission, such as access to some external API data. This app does not request Gmail-reading access in its shown sign-in flow.
Authorization code	In another common OAuth/OIDC flow, a short-lived code is exchanged for tokens. That exchange is absent here.
state / nonce / PKCE	Protocol mechanisms used to bind attempts or protect code exchanges in applicable flows. They are not the same thing as Google's g_state cookie.

The actual chain in your app

```
Google sign-in UI
-> response.credential (Google ID token)
-> POST /auth/google
-> verify_google_token
-> find/create local User
-> create_access_token(local id, auth_version)
-> access_token cookie + JSON
-> protected site APIs
```

Do not swap the two credentials

A Google ID token is verified using Google's keys and your client ID. The site JWT is verified using JWT_SECRET_KEY and then the local user/version. They have different issuers and purposes.

Code / evidence: google_auth.py; frontend/google-sign-in.tsx; supporting OpenID reference

A complete password-login example

Illustrative requests for a local developer account.

Start the configured database and backend first. Run these manually only when you want to create a demo account. The password/address below are example values, and the cookie jar contains a credential after login.

1. Register a user

```
curl -i http://127.0.0.1:8000/auth/register \
-H "Content-Type: application/json" \
-d '{"username": "Hamza42", "email": "hamza@example.com",
    "password": "HamzaLearning2026!"}'
```

Expect 201 with public user fields on the first successful creation. Reusing that username/email returns 409. Registration does not create a login cookie.

2. Log in and save the cookie

```
curl -i -c /tmp/trivia-demo-cookies.txt \
http://127.0.0.1:8000/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "hamza@example.com",
    "password": "HamzaLearning2026!"}'
```

Expect 200, the JSON token response, and Set-Cookie. The -c option writes the cookie jar; it does not extract the token from JSON.

3. Ask who is signed in

```
curl -i -b /tmp/trivia-demo-cookies.txt \
http://127.0.0.1:8000/auth/me
```

Expect 200 with that account. Calling the same endpoint with no cookie or Bearer header returns 401. GET /me does not require the cookie-write Origin check. A protected POST using a cookie would need an allowed Origin.

What was actually verified for this guide

These examples were checked against source contracts. They were not executed against your database, Google account, or SMTP service while creating this documentation.

Code / evidence: Request schemas and route decorators in router.py; session_cookie.py; current_user.py

Follow a recovery link from email to API

The URL fragment and the POST body have different jobs.

```
# Configured frontend destination (illustration)
PASSWORD_RESET_URL=http://127.0.0.1:3000/login?mode=reset-password

# Email link created by forgot_password
http://127.0.0.1:3000/login?mode=reset-password#token=RANDOM_TOKEN
```

1. A visitor submits {"email":"hamza@example.com"} to POST /auth/forgot-password. An eligible account gets a background email attempt; the frontend receives generic 202.
2. The browser opens the frontend URL. The #token fragment is not part of the HTTP request sent to fetch that page. Frontend JavaScript can read it from window.location.hash.
3. captureToken stores the value in memory and removes the fragment from the visible URL. The page asks for a new password and a matching confirmation.
4. The frontend submits the token explicitly in JSON to POST /auth/reset-password. The backend hashes it to locate the stored digest, validates expiry, and atomically consumes it.
5. On success, the frontend clears its token/password state and opens /login?passwordReset=success. The user signs in with the new password; previous JWT versions fail.

```
POST /auth/reset-password
Content-Type: application/json

{"token": "ACTUAL_43_CHARACTER_TOKEN_FROM_THE_LINK",
 "password": "MyNewLearningPassword2026!"}
```

The token text above is a descriptive placeholder. It is not a valid 43-character recovery token and must be replaced for a real request.

Where sensitive values travel

The raw token exists in the email, the browser URL/memory, and the reset POST body. Only its digest is stored in users. The fragment reduces automatic URL transmission to the frontend server, but JavaScript on that page can still access the token.

Code / evidence: password_reset.py; frontend/login-page.tsx; docs/password-reset.md

Configuration and local startup

Know when each setting is read and what failure means.

Setting	Where / when used	If unavailable
JWT_SECRET_KEY	security.py, at module import. Private HS256 signing key.	RuntimeError prevents startup.
GOOGLE_CLIENT_ID	verify_google_token, for each attempt. Public application ID.	Google sign-in returns 503.
SMTP_EMAIL / SMTP_PASSWORD	send_email, when the background task runs.	Log an error; return without sending.
PASSWORD_RESET_URL	forgot_password, before account eligibility. Full frontend URL; no fragment/credentials.	Missing/malformed config returns 503 for everyone.
NEXT_PUBLIC_API_BASE_URL	Frontend build/runtime configuration usage in api.ts. Public API location.	Defaults to local API port 8000.
NEXT_PUBLIC_GOOGLE_CLIENT_ID	GoogleSignIn component. Public client ID.	Component shows Google sign-in unavailable.
Database URL	Hardcoded in dataProcessing/database.py; migration config also needs to point to the intended database.	Database operations fail if the configured database is unavailable.

The startup commands already described by the project

```
cd backend
uv sync
uv run alembic upgrade head
make run
```

These commands install dependencies, apply migrations, and run the development server. PostgreSQL must already be running with the configured host/port/database. The current source points to 127.0.0.1:54320 and trivia_db. make run loads backend/.env; no authentication function opens that file directly.

Snapshot-specific documentation gap

make test and database.md refer to unittest discovery under tests. backend/tests is not present here, so that command is not evidence of available test coverage. Database initialization or migrations were not run to produce this PDF.

Code / evidence: security.py; google_auth.py; mailer.py; password_reset.py; Makefile; database.py

Read the status before debugging the UI

Different failures mean different next steps.

Status / symptom	Likely meaning in this code	Useful check
201	Successful registration.	Expect public user fields, then log in.
200	Successful login, Google login, /me, or reset.	Inspect the operation's contract: not every 200 creates a token.
202	Forgot-password request accepted.	Check eligibility/cooldown and mail configuration if no message arrives.
400	A well-shaped reset token is invalid, expired, or already consumed.	Use the latest eligible email link and ensure the full token reached the API.
401	Password failure or rejected Google/site credential.	Check the endpoint, token expiry, expected issuer, database user, and version.
403	Cookie-authenticated protected write has a disallowed/missing Origin.	Match scheme, hostname, and port exactly to the allowlist.
409	Username/email/identity conflict.	Use another identity or sign in with the existing method.
422	Request fields failed schema validation.	Read detail[].loc and msg; correct fields before retrying.
503	Google verification/configuration unavailable, or bad reset URL config.	Check which endpoint failed and its configuration/dependency.
Network error / 5xx	Transport failure or unhandled backend problem.	Check backend/database availability and logs; do not label every failure "wrong password."

Some database or hashing failures propagate rather than being mapped to one of the explicit authentication errors. AuthRoute catches request-validation errors only. A browser CORS error can also prevent JavaScript from seeing the backend response, even if the server received the request.

Code / evidence: All authentication HTTPException branches and route status codes

“I see a cookie, but I am not logged in”

Presence is only the first step in validation.

1. Check the host where the cookie is stored. A cookie set for localhost is not automatically a cookie for 127.0.0.1. Use one hostname consistently.
2. In the Network panel, inspect the actual /auth/me request. apiFetch includes credentials, but a different raw fetch call might not.
3. Check the response code. 401 means this request did not pass the site’s authentication checks. A cookie can contain an expired, tampered, old-version, or otherwise unacceptable token.
4. Look for an Authorization header. Parsed Bearer credentials take precedence over the cookie; an old header token may be the token being rejected.
5. Check whether the backend secret changed, the user was removed, or a password reset incremented auth_version. All can explain rejection.
6. If /me succeeds but navigation reaches a 404, check whether the target page exists. /join is the current default, and its page is absent from this checkout.

“Why does document.cookie not show access_token?”

It is HttpOnly. The browser may show it in DevTools while hiding its value from page JavaScript. That is expected. /auth/me is the frontend’s way to ask whether the session is accepted.

“Does g_state tell my app that I am logged in?”

No authentication function here reads g_state. Google uses it for its own sign-out/automatic-sign-in behavior. It neither verifies the local JWT nor replaces the access_token cookie. Your current Google component also does not implement site logout.

Check a protected page and a protected API separately

A frontend guard decides what the visitor sees. The backend dependency decides whether a data request may proceed as a particular user. Both need the right behavior; hiding a page does not secure its APIs.

Code / evidence: api.ts; current_user.py; session_cookie.py; frontend/login-page.tsx

What this version does—and what it does not yet do

Use the implemented guarantees without assuming extra features.

Present in this code	Scope / remaining limitation
Persistent users and recovery state	PostgreSQL retains state across process restarts; configuration and database availability still matter.
JWT expiry + auth_version checks	Password resets revoke older versions on subsequent checks. No refresh-token flow or per-device session table.
HttpOnly cookie helper	Both sign-in methods use it. The token is also returned in JSON. No logout endpoint clears the cookie.
Password recovery	One-use digest and 15-minute expiry; per-account 60-second issue cooldown. No IP/global rate limiter in this folder.
Password verification protections	Dummy hash and post-verification refresh. No login attempt limiter or MFA implementation here.
Email syntax validation	Normal registration does not prove mailbox ownership with a verification email.
Google subject verification	No automatic account linking. Existing Google user email is not updated by this route.
Origin checks and local CORS	Origin check lives in the protected-user dependency; it is not applied globally to all public login/reset endpoints.
Background email	No durable queue, retry policy, or delivery guarantee.
Login-page session check	A shared guard for all protected frontend pages is still future work.

Future development could add logout semantics, device-specific sessions, request limits, confirmed registration email ownership, durable mail jobs, account linking, and deployment origin configuration. Each needs a defined behavior and tests; none should be presented as already present.

This is an explanation of observed code behavior, not a claim that a running deployment was audited. The PDF was checked for coverage and layout; backend integration tests were not executed.

Code / evidence: Complete inspected source snapshot

One person, one account, many requests

A complete story using fictional times and user ID 42.

Time	Action	State and result
10:00	Hamza registers.	User 42 is committed with a hash and auth_version=0. Registration returns 201; no cookie yet.
10:01	Hamza logs in.	The password matches. JWT A has sub="42", ver=0, expiry 10:31. It is sent in JSON and an HttpOnly cookie.
10:05	The frontend calls /auth/me.	Signature/time/claim checks pass. User 42 exists and version 0 matches. The endpoint returns public account fields.
10:06	The backend restarts.	Users remain in PostgreSQL. A still works if the same secret/database are used and its other checks pass.
10:10	Hamza requests password recovery.	A random token is generated; only its digest is stored. Expiry is 10:25; email runs after the 202 response.
10:10:20	Another recovery request arrives.	It is within the 60-second cooldown. The same generic 202 returns without issuing another token.
10:12	Hamza submits the valid reset token.	New hash, cleared digest/expiry, and version 1 commit together. He is asked to sign in again.
10:13	A device tries JWT A.	Signature may still pass and exp has not arrived, but ver=0 differs from 1. /me returns 401.
10:14	Hamza logs in with the new password.	JWT B carries ver=1 and expires at 10:44. The cookie is replaced by the new token.
10:45	The device uses JWT B.	The JWT has expired. No refresh endpoint exists, so another sign-in is needed.

Explain the boundary

Registering, logging in, checking /me, and resetting the password are separate requests. Each has its own inputs and response. The browser keeps sending the cookie; the backend keeps deciding whether it is acceptable.

Code / evidence: Illustrative timeline derived from router.py, security.py, password_reset.py, and current_user.py

Can you explain the system now?

Answer before opening the next two pages.

1. A visitor creates an account, then calls `/auth/me` without any credential. What status follows, and which function decides it?
2. A browser has an `access_token` cookie. Why is cookie presence insufficient to show private data?
3. What do `SecretStr`, password hashing, and JWT signing each do? Which one can be unwrapped?
4. Why can two equal passwords produce different hashes?
5. What is the difference between a Google token's `sub` and your site token's `sub`?
6. A normal account exists, and Google returns the same email for a new `google_sub`. What happens?
7. Why does `get_current_user` check a database version after verifying a JWT signature?
8. What happens when a valid cookie and an invalid parsed Bearer header are sent together?
9. Why does `reset_password` query before hashing and conditionally update after hashing?
10. Why can two concurrent registration requests pass the early lookup but only one commit?
11. A forgot-password request returns 202, but no email arrives. Name three explanations.
12. What is different about a malformed reset token and a correctly shaped but expired token?
13. Why is `send_email` scheduled after the response instead of run directly in the route?
14. What does the `active=True/False` cleanup in the frontend session effect prevent?
15. Why is redirecting a visitor to `/login` insufficient protection for a private backend endpoint?
16. Does restarting the backend revoke every session? What conditions determine the answer?

Try a trace without memorizing

Open `get_current_user` and explain every early exit aloud. Then draw the forgot-password and reset-password flows from memory and compare them with the guide.

Code / evidence: Exercises based on the current implementation

Answers • identity and sessions

Questions 1–8

1. **401.** register creates a database row but does not issue a JWT. `get_current_user` sees no credential and raises `Unauthorized()`.
2. **The value must still be accepted.** It may be expired, edited, signed by the wrong secret, carry malformed claims, refer to a missing user, or have an old ver. `/auth/me` performs the checks.
3. **SecretStr masks normal display and can be unwrapped.** Password hashing produces a verifier for candidate passwords. HS256 signing makes unauthorized token changes detectable; it does not hide the JWT payload.
4. **Different salts.** The hashing library stores enough algorithm/salt/settings information for verification. Equal source passwords do not require equal stored hashes.
5. **Different identity systems.** Google's sub identifies the Google account and is stored as `google_sub`. The site JWT sub is the local integer user ID encoded as text.
6. **409, with an instruction to use the existing sign-in method.** The route does not automatically link accounts by matching email.
7. **Revocation before expiry.** A password reset increments `auth_version`, so a still-signed and unexpired older token fails the current-version check.
8. **The header token wins and fails.** The dependency chooses parsed Bearer credentials before looking at the cookie. It does not retry with the cookie after token verification fails.

Connect your answer to a function

Questions 1, 2, 7, and 8 meet in `get_current_user`. Question 3 spans `schemas.py` and `security.py`. Question 6 is a new-account branch in `google_login`. Understanding their relationship is more useful than remembering only their names.

Code / evidence: Answer key derived from the source

Answers • concurrency and recovery

Questions 9–16

9. **The first query avoids wasted hashing; the final UPDATE makes the decision.** The token can expire or be consumed while hashing runs. The UPDATE rechecks it and changes state atomically.
10. **A race can occur before commit.** Both requests see no user and begin hashing. Database uniqueness prevents both normalized identities from being inserted successfully; create_user maps the unique violation.
11. **Several explanations fit 202.** No matching account, a Google-only account, cooldown, missing SMTP settings, delivery failure, or a process interruption after the response. The public message intentionally does not disclose eligibility.
12. **422 versus 400.** Bad length/characters fail ResetPasswordData validation. A correctly shaped token reaches digest/expiry checking, which returns 400 when no eligible state matches.
13. **The HTTP caller need not wait for SMTP.** BackgroundTasks delays that work until after the response. It does not guarantee delivery or provide durable retries.
14. **It ignores a late response after the effect was cleaned up.** It prevents an obsolete fetch result from updating that component/redirecting through that effect. It does not cancel the network request itself.
15. **The browser is controlled by the caller.** They can bypass the screen and call an API. The backend must validate credentials and check ownership/permissions on protected operations.
16. **Not automatically.** PostgreSQL retains user/version state. A token can work after restart when the same signing secret is used and the user, version, claims, and expiration still pass.

A final practical explanation

Try explaining this to another person: “The browser stores a signed token in an HttpOnly cookie. Every protected request verifies that token and looks up its user/version. The frontend uses that API result to choose a page. Resetting a password changes the database version, so old tokens stop passing.” Then name the functions that perform each step.

Code / evidence: Answer key derived from the source

Keep these words close to the code

A quick reference for the terms used throughout the guide.

Term	Meaning
Authentication / authorization	Establish who the caller is / decide what that caller may do.
Credential	Something presented as proof: a password, Google ID token, or site JWT.
Endpoint / router	An HTTP operation at a path / a group of registered operations.
Schema	Rules describing structured input or output.
Dependency injection	The framework runs a declared helper and supplies its result.
Hash / salt	A one-way result / random variation used by password hashing.
Digest	Here, a SHA-256 result stored instead of a raw recovery token.
JWT / claim	A token format / a named piece of information in its payload.
Bearer	Possession of an accepted token is sufficient to present that credential.
Cookie / HttpOnly	Browser-managed storage and request transport / restricted from page JavaScript reads.
Origin	Scheme, hostname, and port together.
CORS / CSRF / XSS	Browser cross-origin response access / forged cross-site actions / unwanted script execution.
ORM	A library mapping database rows to objects such as User.
Primary key / unique constraint	A row identifier / a database rule preventing duplicate key values.
Transaction / commit / rollback	A database unit of work / persist it / abandon it.
Atomic update	A database state change whose conditions and write form one operation.
Race condition	An outcome influenced by overlapping operations and their timing.
Coroutine / await / thread pool	Async computation / wait for a result / workers for synchronous work.
Migration	A recorded, ordered change to database structure.
Revocation / cooldown	Make a previous credential unacceptable / limit how soon an action repeats.
UTC / Unix timestamp	A shared time standard / seconds measured from the Unix epoch.
SMTP / TLS	Email submission/transport protocol / encryption for a network connection.

Code / evidence: Terminology as used by the current implementation

Every function • source index

Entries 1–15 of 29 • line numbers refer to the reviewed snapshot.

File	Definition	Lines	Walkthrough
current_user.py	unauthorized()	16–21	Page 32
current_user.py	get_current_user()	24–48	Page 32
google_auth.py	google_login()	32–69	Page 35
google_auth.py	verify_google_token()	72–97	Page 34
mailer.py	send_email()	10–28	Page 40
password_reset.py	ResetPasswordData.validate_token()	38–42	Page 36
password_reset.py	ResetPasswordData.validate_password()	46–47	Page 36
password_reset.py	forgot_password()	50–86	Page 37
password_reset.py	reset_password()	90–121	Page 38
repository.py	find_user_by_id()	19–22	Page 30
repository.py	find_user_by_email()	25–26	Page 30
repository.py	find_user_by_google_sub()	29–30	Page 30
repository.py	find_user_by_username()	33–34	Page 30
repository.py	find_existing_user_id()	37–41	Page 30
repository.py	create_user()	44–58	Page 31

The walkthrough column links to its explanation in this document. Each function's inputs, work, result/errors, and purpose appear on that page. Function locations were extracted with Python's AST parser, including methods and the nested `safe_handler`.

Code / evidence: Automatically checked against all authentication/*.py definitions

Every function • source index

Entries 16–29 of 29 • line numbers refer to the reviewed snapshot.

File	Definition	Lines	Walkthrough
router.py	register()	30–47	Page 25
router.py	login()	51–67	Page 26
router.py	me()	71–76	Page 26
schemas.py	validate_password_encoding()	17–22	Page 24
schemas.py	RegisterData.validate_username()	37–42	Page 24
schemas.py	RegisterData.validate_password_complexity()	46–53	Page 24
security.py	hash_password()	23–24	Page 27
security.py	verify_password()	27–31	Page 27
security.py	create_access_token()	34–48	Page 28
security.py	decode_access_token()	51–76	Page 28
security.py	authenticate_user()	81–101	Page 29
session_cookie.py	set_access_cookie()	14–27	Page 33
validation_route.py	AuthRoute.get_route_handler()	12–30	Page 41
validation_route.py	AuthRoute.get_route_handler.safe_handler()	15–28	Page 41

The walkthrough column links to its explanation in this document. Each function's inputs, work, result/errors, and purpose appear on that page. Function locations were extracted with Python's AST parser, including methods and the nested `safe_handler`.

Code / evidence: Automatically checked against all authentication/*.py definitions

Further reading • official references

Links 1–7

Implementation explanations come from your local source. These primary/maintainer references support the general concepts and library behavior. Reviewed 6 October 2026; documentation versions may differ from your installed lockfile.

FastAPI dependencies

How Depends supplies a result to route arguments.

<https://fastapi.tiangolo.com/tutorial/dependencies/>

FastAPI async and concurrency

Async work, waiting, and synchronous functions.

<https://fastapi.tiangolo.com/async/>

FastAPI background tasks

Work scheduled after the HTTP response.

<https://fastapi.tiangolo.com/tutorial/background-tasks/>

Pydantic SecretStr

Masked representations and explicit access to secret values.

<https://docs.pydantic.dev/latest/api/types/#pydantic.types.SecretStr>

pwdlib guide

Recommended password hashing and verification.

<https://frankie567.github.io/pwdlib/guide/>

PyJWT usage

JWT signatures, claim checks, and expiration.

<https://pyjwt.readthedocs.io/en/stable/usage.html>

JSON Web Token specification

The JWT format and registered claims.

<https://www.rfc-editor.org/rfc/rfc7519>

Code / evidence: Source links are clickable in the HTML and PDF.

Further reading • official references

Links 8–13

Implementation explanations come from your local source. These primary/maintainer references support the general concepts and library behavior. Reviewed 6 October 2026; documentation versions may differ from your installed lockfile.

SQLAlchemy async ORM

AsyncSession and async database operations.

<https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>

Google ID token verification

Audience, issuer, signature, and Google identity checks.

<https://developers.google.com/identity/gsi/web/guides/verify-google-id-token>

Google automatic sign-in/sign-out

Google Identity Services sign-out state and g_state.

<https://developers.google.com/identity/gsi/web/guides/automatic-sign-in-sign-out>

OpenID Connect explained

The identity layer and the job of an ID token.

<https://openid.net/developers/how-connect-works/>

MDN Set-Cookie

Cookie attributes, host scope, and HttpOnly.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

Python secrets

Cryptographic random token generation.

<https://docs.python.org/3/library/secrets.html>

Code / evidence: Source links are clickable in the HTML and PDF.

Source snapshot and coverage

A guide to this version, with a clear boundary around the claims.

Reviewed the supplied 27-page English PDF, all ten current authentication Python modules, the three text/HTML documentation files, and the backend/frontend integration points cited in the walkthrough. The older Darija PDF is catalogued as a companion reference; it is not used as the authority for current behavior.

Python module	Lines	SHA-256 prefix
current_user.py	48	c8195b1eef5ece57
google_auth.py	97	b992db0111501733
mailer.py	28	994372a1743ec051
password_reset.py	122	53a79fdf7d22a2a0
repository.py	58	fcf3189934faeb3a
router.py	76	60d4b8b04f0c5ec0
schemas.py	68	4d7a968b76255c07
security.py	101	0ee224d751e47457
session_cookie.py	27	35be93e637a64a81
validation_route.py	30	d9a9c802c1779e0c

Coverage check: **29 function definitions** and **9 classes** are catalogued. An AST check verifies that every defined Python function has a walkthrough entry. The manual HTML page's four callbacks/handlers are explained separately.

Source revision at review: c914099. Line numbers and hashes refer to the files used to create this edition. No runtime application code was changed for this documentation. Rendering checks cover PDF page count, text presence, page overflow, internal links, and visual samples.

Use the code to verify the explanation

When the implementation changes, revisit the associated diagram and function page. A useful guide should tell you what the code currently does and why, without turning future plans into present guarantees.

Code / evidence: Prepared 6 October 2026 · current checkout · no production credentials included